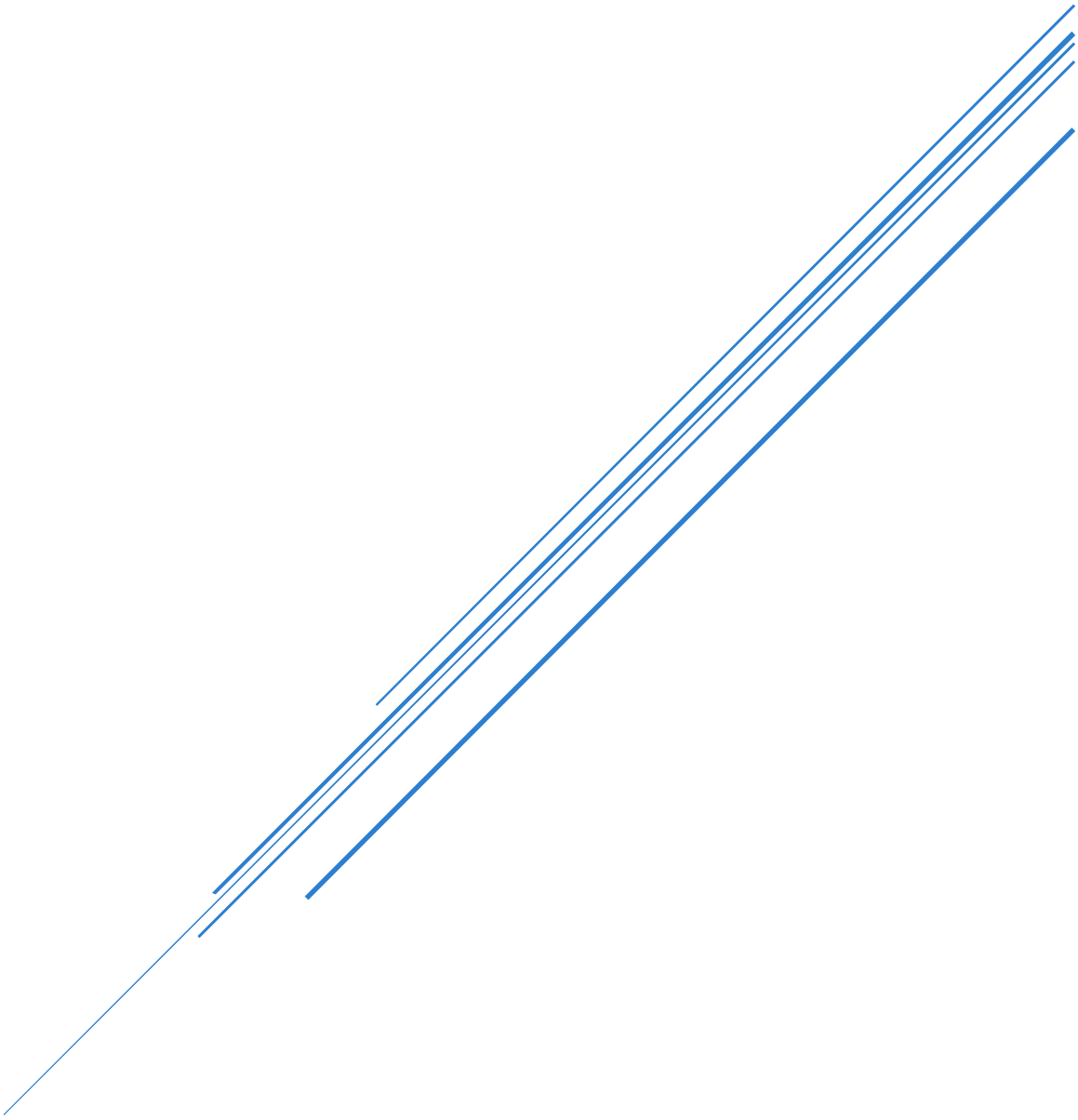
EDGE AI DEPLOYMENT PIPELINE FOR RESOURCE-CONSTRAINED EMBEDDED SYSTEMS



M. Adil Nadeem – AI Intern
PSAU Research & Development Center

Table of Contents

Table of Figures

List of Tables

1 Introduction to Edge AI

1.1 What is Edge AI?

Edge AI refers to the deployment and execution of artificial intelligence and machine learning models directly on physical, local devices (such as microcontrollers, single-board computers, edge gateways, or embedded NPUs) rather than relying on centralized cloud data centers or remote servers. Data generated by local sensors (e.g., cameras, microphones, accelerometers) is processed directly on the device where it is captured, enabling localized perception, inference, and action.

1.2 Why Edge AI?

Traditional cloud-based AI relies on transmitting raw sensor data over network connections to cloud servers for processing, which introduces several operational challenges:

- **Network Latency:** Data transmission creates round-trip delays that are unsuitable for real-time or time-critical applications.
- **Bandwidth & Connectivity Costs:** Streaming continuous raw data (especially video or high-frequency sensor streams) requires significant bandwidth and continuous cloud connectivity.
- **Data Privacy & Security:** Transmitting raw, sensitive personal or environmental data over networks exposes it to potential security risks and regulatory challenges.

Edge AI addresses these challenges by handling inference locally, allowing applications to act instantly on sensor inputs without requiring a constant cloud connection.

1.3 Advantages and Trade-offs of Edge AI

1.3.1 Advantages

- **Low Latency:** Eliminates network round-trips, allowing real-time decision-making.
- **Bandwidth Efficiency:** Minimizes network traffic by processing raw data locally and transmitting only compact results or alerts, if needed.
- **Enhanced Privacy & Security:** Raw sensor data remains on the local device, reducing security exposure and simplifying privacy compliance.
- **Offline Operation & High Reliability:** Operates reliably even in remote or air-gapped environments without stable network connectivity.
- **Lower Operating Costs:** Reduces ongoing cloud server processing and network data transfer expenses.

1.3.2 Trade-offs & Challenges

- **Resource Constraints:** Embedded targets have limited compute power, memory (RAM), and storage compared to cloud servers.
- **Power & Thermal Budget:** Edge hardware often runs on battery or limited energy sources, requiring strict performance-per-watt efficiency.
- **Deployment & Model Complexity:** Requires specialized model optimization (e.g., quantization, pruning) and conversion steps to fit hardware limits.
- **Maintenance & Updating:** Updating firmware and models across distributed fleets of physical devices is more complex than updating a centralized cloud service.

2 Edge AI Deployment Pipeline

Edge AI runs inference directly on the device that captures the data, instead of sending it to a remote server — sensing, inference, and action all happen on-device, with no cloud round-trip required. This avoids the latency, bandwidth cost, and privacy exposure of cloud inference, at the cost of the limited compute, memory, and power budget of embedded hardware (summarized as BLERP: Bandwidth, Latency, Energy, Reliability, Privacy). Every stage covered in this document fits into a single reusable pipeline, split into a build-time (offline) phase and a run-time (on-device) loop.

2.1 Build-Time Pipeline

Figure 1: Build-Time Pipeline

What happens once, before anything is flashed to a device:

1. **Problem Definition:** classification or regression, and whether the task actually needs to run at the edge
2. **Data Collection & Preprocessing:** gather, clean, and prepare data from public sources or a custom capture pipeline
3. **Model Training:** fit a model to the training data, optionally fine-tuning a pretrained model
4. **Evaluation:** accuracy, precision, recall, F1, MAE/MSE — measured against held-out data
5. **Optimization:** quantize and prune the model for the target hardware
6. **Conversion:** convert the optimized model into a runtime format the target device can execute
7. **Flash to Device:** deploy the converted binary to the embedded target

> *This build-time sequence is the same general ML pipeline used for any model, with one edge-specific addition: the optimization and conversion steps that make a model small and fast enough to run on constrained hardware.*

2.2 Run-Time Loop

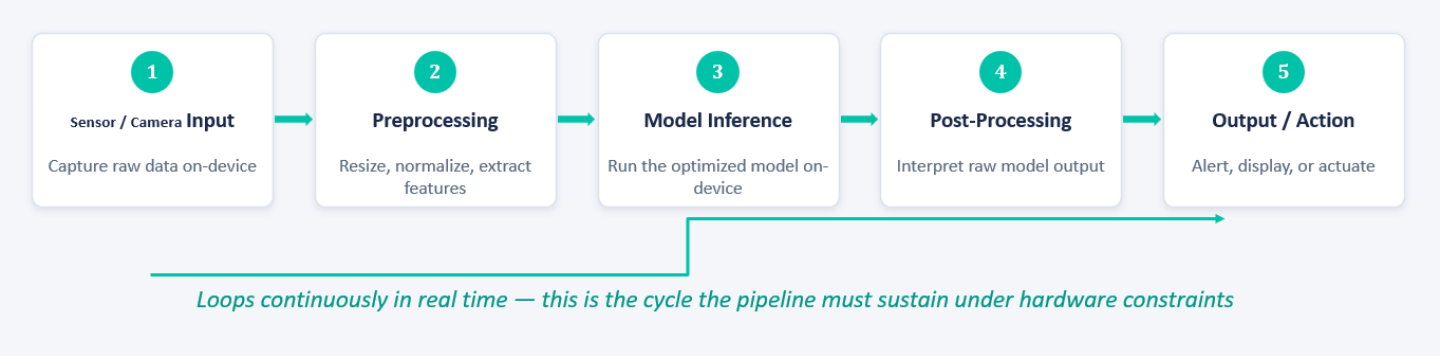


Figure 2: Run-Time Pipeline

What runs continuously, in real time, once the model is on the device:

1. **Sensor / Camera Input:** capture raw data on-device (image, audio, motion, etc.)
2. **Preprocessing:** resize, normalize, and extract features from the raw input
3. **Model Inference:** run the optimized model on-device to produce a prediction
4. **Post-Processing:** interpret the raw model output (e.g., thresholding, label mapping)
5. **Output / Action:** alert, display, or actuate based on the result

This loop is what the pipeline must sustain in real time, under the hardware's compute, memory, and power constraints — which is why the build-time optimization and evaluation stages matter so much.

2.3 Model Conversion Pipeline

Once a model is trained and optimized, it needs to be converted into a format the target runtime can actually execute:

- **Trained model:** the output of training workflow, typically in a TensorFlow or PyTorch format
- **ONNX:** a common interchange format used as an intermediate step between training frameworks and edge runtimes
- **TensorFlow Lite Micro / NCNN:** the target runtime — TFLite Micro for MCU targets like the ESP32, or an ARM-optimized runtime such as NCNN for OS-based boards like the Raspberry Pi and AMB82-Mini
- **Flashed to device:** the converted model is packaged with the application code and flashed to the board, ready for on-device inference

3 Hardware Selection and Analysis

3.1 Hardware Selection Approach

Choosing hardware for embedded AI is not simply about picking the most powerful chip available. The decision effects performance, power consumption, scalability, certification requirements, cost, and the long-term lifecycle of the product. For embedded and edge AI specifically, performance per watt is usually a more useful measure than raw compute power, since these devices are almost always constrained by battery life or thermal limits.

The following factors guide hardware selection for this project:

- **Power consumption** – balancing inference performance against energy and thermal limits
- **Cost** – fitting the target budget per unit
- **Scalability** – leaving headroom for future model updates or heavier workloads

- **Use case** – the required inference speed, accuracy, and type of AI task (e.g., audio, vision)
- **Team expertise** – compatibility with familiar tools and frameworks
- **Lifecycle & supply chain** – long-term component availability

Different workloads call for different tiers of hardware, ranging from low-end microcontrollers for simple sensor tasks up to GPUs/NPUs for real-time vision. The three platforms compared here — ESP32, Raspberry Pi Zero 2 W, and AMB82-Mini — sit in the microcontroller and low-power SoC tier, suitable for TinyML-style workloads such as keyword spotting, gesture recognition, and lightweight image classification rather than full real-time video AI.

3.2 Hardware Specifications and Analysis

Each candidate board was evaluated on its processor, memory, connectivity, AI acceleration support, and cost, since these directly determine which models can realistically run on-device.

3.2.1 ESP32

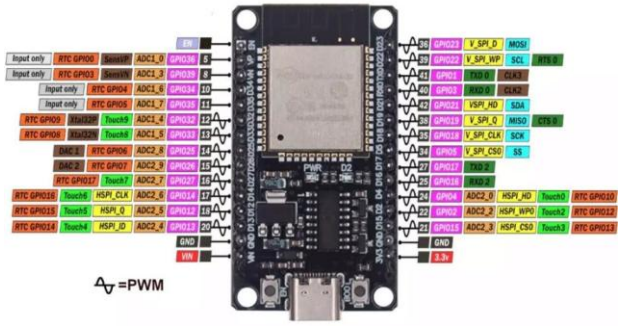


Figure 3: ESP32

The ESP32 is a low-cost, dual-core microcontroller from Espressif that has become a default choice for TinyML because of its balance of cost, connectivity, and community/framework support (TensorFlow Lite Micro, Edge Impulse).

- Processor: Dual-core Xtensa LX6, up to 240 MHz
- RAM: ~520 KB SRAM on-chip (no external RAM by default)
- Storage: Typically, 4 MB external flash for firmware and model weights
- Wireless: Wi-Fi (802.11 b/g/n) and Bluetooth/BLE
- AI acceleration: None (pure CPU inference); relies on quantized models via TensorFlow Lite Micro / ESP-DL
- Approx. cost: $5–$10 depending on module/board

Its limited RAM restricts models to a few hundred kilobytes once quantized, making it best suited to simple classification or keyword-spotting tasks rather than vision-heavy workloads.

3.2.2 Raspberry Pi Zero 2 W

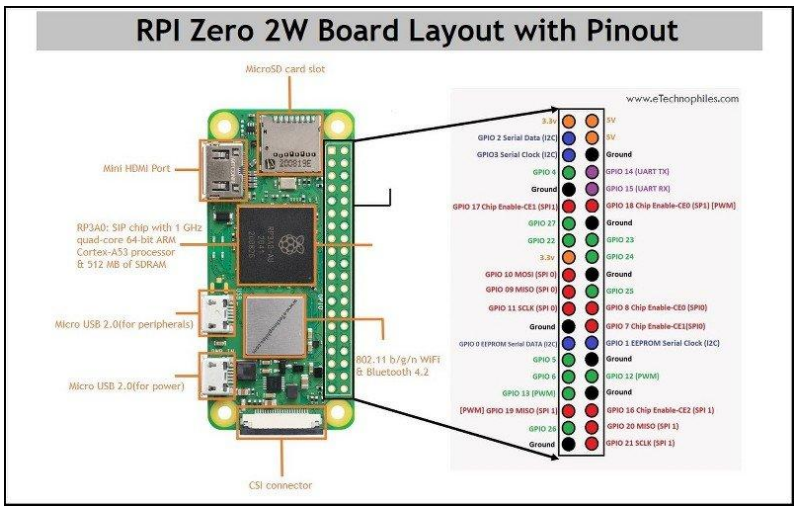


Figure 4: Raspberry Pi Zero 2 W

The Raspberry Pi Zero 2 W is a full single-board computer rather than a microcontroller, running a Linux OS. This gives it access to standard ML libraries and development tools, at the cost of higher power draw than a bare MCU.

- Processor: 1 GHz quad-core 64-bit ARM Cortex-A53
- RAM: 512 MB SDRAM
- Storage: Requires a separate micro-SD card (32 GB minimum recommended)
- Wireless: 2.4 GHz 802.11 b/g/n Wi-Fi, Bluetooth 4.2, BLE
- Ports: Mini HDMI, micro-USB OTG, CSI-2 camera connector
- Power: 5 V via micro-USB (needs a 5 V / 2.5 A adapter)
- AI acceleration: None (CPU-only); benefits most from optimized runtimes such as TensorFlow Lite or ONNX Runtime
- Approx. cost: ~$15

Its quad-core CPU and full OS make it capable of running larger quantized models and standard Python-based ML pipelines, at the cost of more power draw and a slower boot/setup process than a microcontroller.

3.2.3 AMB82-Mini

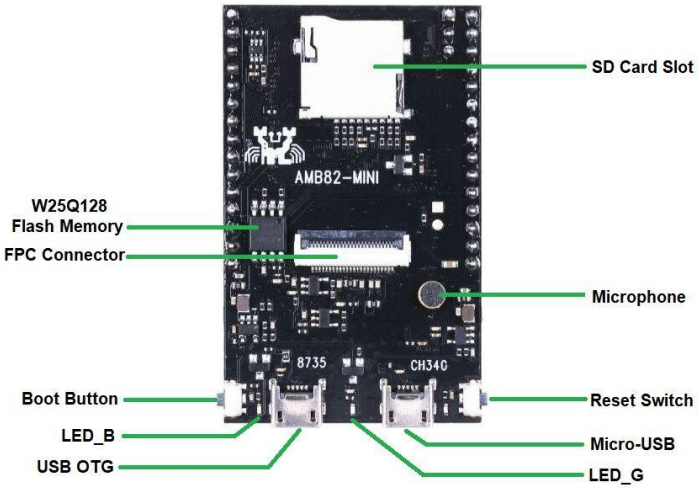


Figure 5: AMB82-Mini

The AMB82-Mini is a purpose-built AIoT camera board from Realtek/Seeed Studio, combining a Wi-Fi/Bluetooth SoC with a dedicated neural network accelerator — making it the only one of the three boards with hardware AI acceleration.

- SoC: Realtek RTL8735BDM
- Processor: Arm v8M MCU, up to 500 MHz
- AI accelerator: Built-in NPU (≈ 0.4 TOPS "Intelligent Engine") for on-chip neural network inference
- Memory: 128 MB internal DDR2 + 16 MB external SPI NOR flash
- Wireless: Dual-band Wi-Fi + Bluetooth BLE 5.1
- Camera & video: Built-in 1080p sensor (**JX-F37**) with H.264/H.265 hardware encoding
- Programming: Arduino IDE, FreeRTOS SDK; supports TensorFlow Lite models
- Approx. cost: ~$30

The dedicated NPU and hardware video encoder make it well suited to on-device computer vision (object detection, face detection) at a fraction of the power draw of a full SBC — though at a higher unit cost than the ESP32.

3.3 Hardware Comparison Table

Spec	ESP32	Raspberry Pi Zero 2 W	AMB82-Mini
Processor	Dual-core Xtensa LX6 @ 240 MHz	Quad-core ARM Cortex-A53 @ 1 GHz	Arm v8M MCU @ 500 MHz
RAM	~520 KB SRAM	512 MB SDRAM	128 MB DDR2 (internal)
AI acceleration	None (CPU only)	None (CPU only)	Built-in NPU (~0.4 TOPS)
Camera support	Via add-on module (e.g., OV2640)	Via CSI-2 connector	Built-in 1080p sensor (JX-F37)
Connectivity	Wi-Fi + Bluetooth/BLE	Wi-Fi + Bluetooth 4.2/BLE	Dual-band Wi-Fi + BLE 5.1
OS / dev environment	Bare-metal / RTOS (Arduino, ESP-IDF)	Full Linux (Raspberry Pi OS)	Arduino IDE / FreeRTOS
Best suited for	Simple audio/sensor classification, keyword spotting	General-purpose edge ML, Python pipelines	On-device vision (detection/recognition)
Approx. cost	$5–$10	~$15	~$30

Table 1: Hardware Comparison

4 Model Types and Characteristics

Machine learning tasks generally fall into **regression** (predicting a continuous value, e.g., price) or **classification** (assigning inputs to predefined categories, e.g., spam detection), and the right model choice depends on which of these the task requires. Building a model is only the first step — a model can score well on paper and still perform poorly in practice, so this section covers the model types, characteristics, and evaluation criteria used to judge whether a model is actually fit for purpose.

4.1 Types of AI Models

Type	How it learns	Example techniques	Typical use
Supervised learning	From labelled data (inputs + correct outputs)	Linear/logistic regression, decision trees	Credit scoring, spam detection, forecasting
Unsupervised learning	From unlabeled data; finds structure on its own	Clustering, dimensionality reduction	Customer segmentation, anomaly detection
Reinforcement learning	Trial and error via rewards/penalties	Reinforcement learning algorithms	Supply-chain optimization, autonomous vehicles
Deep learning	Multiple layers, each capturing more complex patterns	CNNs, RNNs	Image recognition, time-series forecasting, voice assistants
Generative AI	Learns patterns to generate new content	GANs, LLMs	Text generation, summarization, code generation

Table 2: Types of AI Models

4.2 Model Characteristics

4.2.1 Model Parameters

Parameters are the **learned internal values** — coefficients in regression, weights and biases in neural networks — that **a model adjusts during training so its outputs match expected results**. In unsupervised learning, parameters (e.g., cluster centroids in k-means) are instead learned by optimizing how well the model explains structure in the data itself.

4.2.2 Model Size

Model size refers to the **number of parameters a model holds**, which directly **determines** its **memory footprint**, its **compute cost per prediction**, and — loosely, via scaling laws — its **capability**. For scale: Llama 3 8B holds about 8 billion parameters, GPT-3 held 175 billion, and Moonshot's Kimi K3 (the largest open-weight model as of 2026) holds 2.8 trillion. Larger models need proportionally more memory to store and more compute to run, which is a central constraint for edge deployment.

4.2.3 Computational Complexity and Generalization

Model complexity reflects how much **capacity a model has to capture patterns**, generally scaling with its parameter and feature count. Overly simple models **underfit** (missing real patterns); overly complex ones **overfit** (memorizing the training data instead of generalizing). A well-chosen model balances complexity against generalization — an especially important trade-off for edge deployment, where unnecessary complexity also costs scarce memory and compute.

5 Dataset Sources and Custom Dataset Development

5.1 Dataset Sources

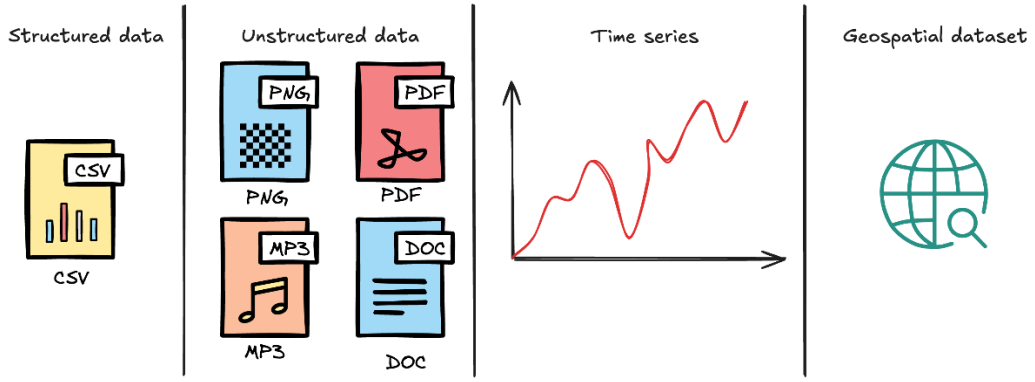


Figure 6: Dataset Sources

Data used for training generally falls into a few categories, and identifying the type early helps determine the right storage, preprocessing, and modeling approach:

- **Structured:** tabular data such as spreadsheets or CSVs, easily queried with SQL
- **Unstructured:** text, images, audio — typically needs NLP or computer-vision preprocessing
- **Time-series:** observations ordered by time, used for forecasting and sensor monitoring
- **Geospatial:** data tied to geographic coordinates, used in GIS and navigation

Open datasets are trending toward larger, multimodal collections that are increasingly cloud-hosted (e.g., AWS Open Data, Hugging Face), making them easier to access and process at scale.

Commonly used dataset sources:

- **Kaggle:** broad collection of datasets across domains, useful for experimentation and benchmarking
- **UCI ML Repository:** curated datasets widely used for classical ML benchmarking
- **Google Dataset Search:** searches across many online dataset repositories at once
- **AWS Open Data Registry:** free cloud-hosted datasets (climate, genomics, satellite imagery, etc.)
- **Hugging Face Datasets:** large hub of ready-to-use datasets with built-in preprocessing tools

A broader list of dataset repositories (government portals, domain-specific hubs, etc.) is included in the References section.

5.2 Dataset Selection

Before adopting any dataset, the following checks help ensure it is fit for purpose:

- Check licensing and privacy terms before use
- Validate data quality — look for duplicates, noise, missing values, and class imbalance
- Start small — use a subset if the full dataset exceeds available compute/storage
- Define evaluation metrics appropriate to the task before training begins

A dataset is only useful for training if it is “model-ready” — meaning it is accurate (correctly labeled), relevant to the real-world use case, complete (minimal missing data), and compliant with privacy/regulatory requirements. Existing public datasets should be used where they fit; a custom dataset becomes necessary when the target application has specialized data needs that public datasets don't cover.

5.3 Custom Dataset Training and Workflow

A custom dataset is a curated collection built specifically for a given task, used when public datasets don't adequately represent the target problem (e.g., a specific object, environment, or sensor setup).

Figure 7: Custom Dataset Training Workflow

5.3.1 Building a custom dataset

- **Data collection** – gather diverse, representative samples, including edge cases
- **Cleaning & preprocessing** – remove duplicates, fix inconsistent formats, handle missing values
- **Labeling / annotation** – add accurate labels (classes, bounding boxes, etc.)
- **Organization, storage & version control** – keep the dataset reproducible over time
- **Quality checks** – verify label accuracy and check for bias or distribution issues

For image-based datasets specifically, consistent labeling, data augmentation, and systematic quality verification matter more than raw volume.

5.3.2 End-to-end training workflow

Once data is ready, the general model development workflow follows these stages:

1. **Define the problem** – objective, expected output, and success metric
2. **Collect & prepare data** – gather and label representative data
3. **Preprocess** – clean, normalize, encode, and apply task-specific steps (e.g., image resizing/augmentation)
4. **Split the dataset** – training / validation / test sets, avoiding data leakage
5. **Select a model** – based on task type, data size, and available compute
6. **Train** – minimize a loss function over multiple epochs/batches
7. **Tune hyperparameters** – learning rate, batch size, model depth, etc.
8. **Evaluate** – accuracy/F1/ROC-AUC for classification, MSE/R^2 for regression
9. **Address over/underfitting** – regularization and more data vs. increased model capacity
10. **Deploy** – to the target edge device, considering latency and monitoring
11. **Iterate** – continuously improve using real-world feedback

5.4 Fine-Tuning Pre-Trained Models

A pretrained model has already been trained on a large, general dataset and can be reused or adapted (fine-tuned) for a related, more specific task. This is typically faster and far less resource-intensive than training a model from scratch.

Benefits of starting from a pretrained model:

- **Faster development** — avoids training from scratch
- **Lower resource requirements** — needs less data and compute
- **Transfer learning** — general learned features adapt to a domain-specific task through fine-tuning
- **Reduced risk** — the base model is already tested and benchmarked

Common **places to find** pretrained models: **PyTorch Hub**, **TensorFlow Hub**, **Hugging Face**, **Kaggle**, **GitHub**, **NVIDIA NGC**, and **KerasHub**. Fine-tuning typically means freezing most of the pretrained network's early layers (general features) and retraining the later layers on the target dataset.

6 Model Evaluation Criteria and Performance Metrics

6.1 Evaluation Criteria

6.1.1 Classification Metrics

A **confusion matrix** breaks down a classifier's predictions against ground truth, and is the basis for most classification metrics:

	Actually Positive	Actually Negative
Predicted Positive	True Positive (TP)	False Positive (FP)
Predicted Negative	False Negative (FN)	True Negative (TN)

Table 3: Confusion Matrix

- **Accuracy: correct predictions ÷ total predictions** — a common but incomplete measure of performance on its own
- **Precision = TP / (TP + FP):** of predicted positives, how many were correct; matters when false positives are costly (e.g., flagging legitimate transactions as fraud)
- **Recall = TP / (TP + FN):** of actual positives, how many were caught; matters when false negatives are costly (e.g., missing a disease diagnosis)
- **F1 = 2 × (Precision × Recall) / (Precision + Recall):** harmonic mean of precision and recall; useful for imbalanced datasets where one class is rare

6.1.2 Regression Metrics

- **MAE:** average absolute difference between actual and predicted values — treats **all errors equally** and stays in the target's original units
- **MSE:** average squared difference between actual and predicted values — **penalizes large errors** more heavily, making it sensitive to outliers

6.2 Performance Metrics for Edge Deployment

Unlike conventional model evaluation, where predictive performance is usually the main concern, edge deployment also needs to assess whether a model can run efficiently within an embedded device's limited compute, memory, and energy budget. A systematic review of neural-network deployment on embedded systems identifies latency, throughput, energy consumption, and memory usage as the key inference metrics for edge AI.

- **Inference latency:** time to process one input and produce an output (ms or µs) — lower is faster, and critical for real-time applications with a fixed response-time budget
- **Throughput:** inputs processed per unit time, expressed as inferences per second (IPS) or frames per second (FPS) — higher means more inputs handled in the same time
- **Energy consumption:** energy used per inference, in watts (W) or milliwatts (mW) — lower means better energy efficiency, critical for battery-powered devices
- **Memory usage:** RAM required during inference (KB or MB) — especially important on embedded systems, where available memory is far more limited than on conventional computing platforms

7 Model Optimization

7.1 Why Optimization

Edge devices — especially microcontrollers like the ESP32, which may have 256 KB of RAM or less — cannot run models the way a server or GPU can. Optimization adapts a trained model to fit these constraints while keeping it useful. The main benefits are:

- **Size reduction:** less on-device storage, faster downloads/updates, and lower RAM use during inference
- **Latency reduction:** fewer computations per inference means faster responses
- **Lower power consumption:** critical for battery-powered devices
- **Wider hardware compatibility:** integer-based models can run on hardware with limited or no floating-point support

Without optimization, most modern models are simply too large or too slow to run on microcontroller-class hardware.

7.2 Optimization Techniques

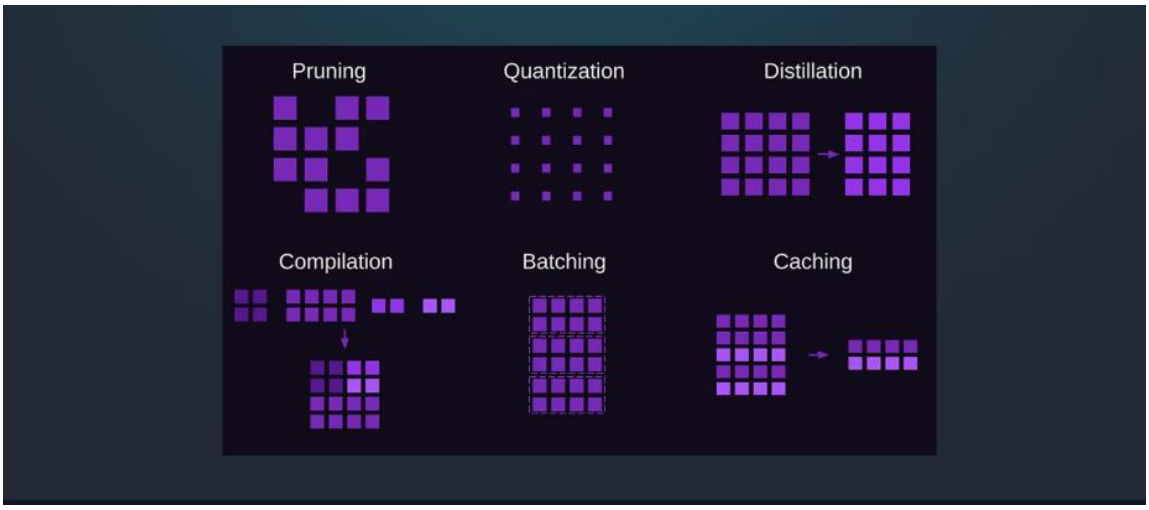


Figure 8: Optimization Techniques

Several complementary techniques are commonly used to make models more efficient. Following table focuses on six of them:

Technique	What it does	Speed	Memory	Quality
Quantization	Reduces the numeric precision of weights/activations (e.g., FP32 → INT8)	↑	↓	slight ↓

Technique	What it does	Speed	Memory	Quality
Pruning	Removes less-important weights/connections, creating a sparser network	↑	↓	slight ↓
Distillation	Trains a smaller “student” model to mimic a larger “teacher” model	↑	↓	slight ↓
Compilation	Compiles the model into instructions optimized for the specific target hardware	↑	≈	≈
Batching	Groups multiple inputs together to process them simultaneously	↑	≈ / ↑	≈
Caching	Stores intermediate computation results to speed up repeated operations	↑	≈	≈

Table 4: Optimization Techniques

> *↑ = improves, ↓ = reduces, ≈ = roughly unchanged.*

On very small devices, quantization, pruning, and distillation are the most impactful because they directly shrink the model. Compilation, batching, and caching mainly improve runtime efficiency of a model that's already been fitted to the hardware.

7.3 Optimization Evaluation

Optimization should always be measured, not assumed. Establishing a baseline before any changes — and re-measuring after — is what makes it possible to judge whether an optimization was actually worth it. Key metrics to track:

- **Inference latency** (time per prediction)
- **Memory usage** (RAM and storage footprint)
- **Power consumption**
- **Accuracy** (compared against the unoptimized baseline)

Because optimization benefits vary a lot between CPUs, GPUs, MCUs, and NPUs, and because some operations may not be supported on the target hardware, every optimized model should be re-validated on the actual target device rather than assumed to behave the same as on a development machine.

7.4 Quantization

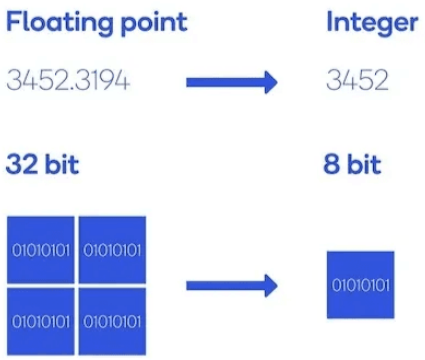


Figure 9: Quantization

Quantization is the primary optimization technique used in this project. It converts model weights and activations from high-precision floating-point formats (e.g., FP32) to lower-precision formats (e.g., INT8), reducing memory and computation requirements with minimal code changes to the model architecture itself.

7.4.1 Advantages and Trade-offs

Advantages

- **Smaller model size** — easier to fit in limited flash/RAM
- **Faster inference** — integer arithmetic is cheaper than floating-point on most MCUs
- **Lower power consumption** — fewer, cheaper operations per inference
- **Broader hardware compatibility** — runs on hardware with limited or no FPU

Trade-offs / cons

- **Accuracy loss** from reduced numerical precision ("quantization error")
- **Hardware-dependent gains** — INT8 isn't automatically faster without optimized integer support
- **Some layers/operations may not support quantization** and need special handling
- **Requires additional validation** — accuracy and performance must be re-tested after quantizing
- Quantization-Aware Training (QAT), when needed, adds **extra training cost**

7.4.2 Process and Workflow

At its core, quantization maps floating-point values to integers using a scale and a zero-point (offset):

```
quantized = round(float / scale) + zero_point
float ≈ (quantized – zero_point) × scale
```

- **Scale:** sets the step size when mapping float values to integers
- **Zero-point:** aligns floating-point zero with the integer range (0 for **symmetric** quantization; non-zero for **asymmetric**)

A typical **INT8 quantization workflow**:

1. Identify which operations/layers are suitable for quantization
2. Try dynamic quantization first (simplest, no calibration data needed)
3. If needed, move to static quantization using a representative calibration dataset
4. Run calibration to determine activation ranges
5. Convert FP32 operations to INT8
6. Evaluate resulting accuracy and performance against the baseline
7. If accuracy loss is too high, fall back to QAT

7.4.3 Approaches

- **Post-Training Quantization (PTQ):** applied after the model is already trained; fastest to implement, but can lose more accuracy. Comes in two flavors — **static** (activation ranges precomputed from a calibration dataset, faster at inference) and **dynamic** (activation ranges computed on the fly during inference, simpler but with runtime overhead)
- **Quantization-Aware Training (QAT):** simulates the effect of quantization during training itself, so the model learns to compensate — recovers more accuracy but requires extra training resources
- **Dynamic Quantization:** quantizes selected components (commonly just the weights) while computing activation ranges at runtime; a good middle ground when a calibration dataset isn't available

For embedded/edge deployment specifically, static PTQ is generally preferred, since inference-time efficiency matters more than the small extra setup effort of calibration.

7.4.4 Numeric Formats

Different numeric formats trade off range, precision, size, and speed:

Type	Description	Typical use
FP32	32-bit floating point — the default training precision	Baseline / training
FP16 / BF16	16-bit floating point — moderate size/speed gain, simple conversion from FP32	GPU inference, mixed-precision training
INT8	8-bit integer — large compression (256 possible values), most common target for edge deployment	MCU / edge inference
INT4	4-bit integer — maximum compression, higher accuracy-loss risk	Extremely constrained devices/LLMs
INT16 activations + INT8 weights	Keeps activations at higher precision than weights	Cases where activations are sensitive to quantization

Table 5: Types of Numeric Formats

Quantization can also be applied to different parts of a model:

- **Weights:** model parameters — quantizing these mainly reduces model size
- **Activations:** intermediate outputs during inference — quantizing these mainly reduces compute cost

And using either **symmetric quantization** (zero-point = 0, simpler and slightly cheaper to compute) or **asymmetric quantization** (non-zero zero-point, used when the value range isn't centered around zero). Weights + activations both in INT8 gives the maximum efficiency gain but carries the highest accuracy risk; quantizing weights only is more conservative and still shrinks model size meaningfully.

8 Case Study: Real-Time Drone Detection on the AMB82-Mini

Custom YOLOv7-tiny model — training, conversion, deployment, and live RTSP verification

This case study documents a complete, reproducible build of a single-class ("drone") object detector: trained from scratch in PyTorch, converted for the AMB82-Mini's on-chip NPU, flashed using the Ameba ObjectDetectionLoop example, and verified live over an RTSP stream in VLC. It is written to stand on its own — every command used is included, so it does not require the original training notebook to follow.

8.1 Project Overview

The goal was to get a custom-trained object detector — not one of the AMB82-Mini's built-in 80-class COCO models — running on-device and streaming live detection results. The target object was a single class: drone.

At a high level, the project followed the same build-time / run-time pipeline used elsewhere in this document:

Pipeline stage	What was done here
Problem Definition	Single-class object detection (“drone”), real-time, on-device
Data Collection	Public Kaggle drone-detection dataset, pre-split train/valid, YOLO format
Model Training	YOLOv7-tiny (PyTorch), trained on Kaggle (GPU)
Evaluation	test.py / detect.py against the validation split
Optimization & Conversion	Reparameterization + AmebaPro2 online model conversion to .nb
Flash to Device	Arduino IDE + Ameba Pro2 SDK, ObjectDetectionLoop example
Run-Time Loop	On-device inference → RTSP stream → verified live in VLC

Table 6: Real-Time Drone Detection on the AMB82-Mini Case Study Pipeline

8.1.1 Hardware and Software Used

- **Board:** AMB82-Mini (Realtek AmebaPro2 / RTL8735B), on-board JX-F37 camera
- **Training:** Kaggle notebook (GPU runtime), PyTorch, YOLOv7 (WongKinYiu/yolov7)
- **Deployment toolchain:** Arduino IDE with the Realtek AmebaPro2 board package (v4.1.0) and its NeuralNetwork library
- **Model conversion:** AmebaPro2 online AI model conversion tool
- **Verification:** VLC media player (RTSP playback)

Figure 10: AMB82-Mini with JX-F37

8.2 Dataset

Training data came from a public Kaggle dataset, already split into training and validation sets in YOLO annotation format (one .txt label file per image, normalized bounding boxes).

Kaggle – YOLO Drone Detection Dataset (muki2003)

Property	Value
Classes	1 (drone)
Format	YOLO (images/ + labels/, normalized .txt per image)
Split	Pre-split train / valid
Input size used	320×320

Table 7: Real-Time Drone Detection on the AMB82-Mini Case Study Dataset Properties

8.3 Training Environment Setup (Kaggle)

Training — and, importantly, the later reparameterization step — needs a GPU and a specific PyTorch behavior that YOLOv7's (older) codebase expects. Kaggle's free GPU notebooks were used for this reason.

> **⚠ Finding: local (no-GPU) environment did not work**
>
> Running the reparameterization step on a local Windows machine without a GPU failed due to a PyTorch dependency conflict. The same steps worked without issue on Kaggle's GPU runtime.

8.3.1 Verify GPU and PyTorch

```
import torch
import sys

print("Python:", sys.version)
print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("CUDA version:", torch.version.cuda)
```

8.3.2 Clone YOLOv7 and Install Dependencies

```
%cd /kaggle/working
!git clone https://github.com/WongKinYiu/yolov7.git

%cd /kaggle/working/yolov7
!pip install -q matplotlib pandas scipy seaborn pyyaml tqdm opencv-python pillow requests psutil
gitpython thop tensorboard
```

8.4 Data Preparation

The dataset was located under Kaggle's read-only /kaggle/input, verified, then copied to the writable /kaggle/working directory before training.

8.4.1 Locate and Verify the Dataset

```
from pathlib import Path

dataset_candidates = list(Path("/kaggle/input").rglob("drone_dataset"))
dataset = dataset_candidates[0]

train_images = dataset / "train" / "images"
train_labels = dataset / "train" / "labels"
valid_images = dataset / "valid" / "images"
valid_labels = dataset / "valid" / "labels"
```

Image and label counts were checked on both splits, and one label file was opened to confirm the normalized YOLO format (class x_center y_center width height).

8.4.2 Copy to a Writable Directory

```
import shutil
from pathlib import Path

working_dataset = Path("/kaggle/working/drone_dataset")
if not working_dataset.exists():
    shutil.copytree(dataset, working_dataset)
```

8.4.3 Create the data.yaml

```
data_yaml = f"""train: {working_dataset / "train"}
val: {working_dataset / "valid"}

nc: 1
names:
  - drone
"""

Path("/kaggle/working/drone_data.yaml").write_text(data_yaml.strip())
```

8.5 Model Configuration

8.5.1 Set the Model to 1 Class

YOLOv7-tiny's default training config targets the 80-class COCO dataset, so the class count was changed to match the single “drone” class:

```
cfg_path = Path("/kaggle/working/yolov7/cfg/training/yolov7-tiny.yaml")
text = cfg_path.read_text()
text = text.replace("nc: 80", "nc: 1")
cfg_path.write_text(text)
```

8.5.2 Disable Experiment-Tracking Prompts

Left enabled, YOLOv7's training script prompts interactively for Weights & Biases setup, which stalls a non-interactive notebook run:

```
import os
os.environ["WANDB_DISABLED"] = "true"
os.environ["WANDB_MODE"] = "disabled"
```

8.5.3 PyTorch Checkpoint-Loading Compatibility

YOLOv7 is an older codebase that expects PyTorch's pre-2.6 torch.load default behavior:

```
import os
os.environ["TORCH_FORCE_NO_WEIGHTS_ONLY_LOAD"] = "1"
```

8.6 Training the Model

Training was run in two stages: a 1-epoch sanity check to confirm the pipeline worked end-to-end, then the real training run.

8.6.1 Sanity Check (1 Epoch)

```
%cd /kaggle/working/yolov7

!python -u train.py \
    --workers 2 \
    --device 0 \
    --batch-size 16 \
    --data /kaggle/working/drone_data.yaml \
    --img 320 320 \
    --cfg cfg/training/yolov7-tiny.yaml \
    --weights "" \
    --name drone-test \
    --hyp data/hyp.scratch.tiny.yaml \
    --epochs 1
```

Two full training approaches were then tried and compared:

8.6.2 Approach A — 100 Epochs, From Scratch

```
!python -u train.py \
    --workers 2 \
    --device 0 \
    --batch-size 16 \
    --data /kaggle/working/drone_data.yaml \
    --img 320 320 \
    --cfg cfg/training/yolov7-tiny.yaml \
    --weights "" \
    --name yolov7-tiny-drone-100 \
    --hyp data/hyp.scratch.tiny.yaml \
    --epochs 100
```

8.6.3 Approach B — 50 Epochs, Fine-Tuned from Pretrained Weights

The official YOLOv7-tiny COCO-pretrained weights were downloaded and used as the starting point for a shorter, fine-tuning run:

```
!curl -L -o yolov7-tiny.pt "https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-tiny.pt"

!python -u train.py \
    --workers 2 \
    --device 0 \
    --batch-size 16 \
    --data /kaggle/working/drone_data.yaml \
    --img-size 320 320 \
    --cfg cfg/training/yolov7-tiny.yaml \
    --weights /kaggle/working/yolov7-tiny.pt \
    --name drone-pretrained-50 \
    --hyp data/hyp.scratch.tiny.yaml \
    --epochs 50
```

> *The fine-tuned model from Approach B (drone-pretrained-50) is the version carried forward through reparameterization, conversion, and deployment in the rest of this document — consistent with the fine-tuning-a-pretrained-model approach described in the main document's dataset section.*

8.7 Validation and Detection Testing

8.7.1 Quantitative Validation

```
!python -u test.py \
    --data /kaggle/working/drone_data.yaml \
    --img 320 \
    --batch 16 \
```

```
    --conf 0.001 \
    --iou 0.65 \
    --device 0 \
    --weights runs/train/drone-pretrained-50/weights/best.pt \
    --name drone-validation
```

8.7.2 Visual Detection Check

detect.py was run across the full validation image set to visually inspect bounding boxes rather than relying on metrics alone:

```
!python detect.py \
    --weights runs/train/drone-pretrained-50/weights/best.pt \
    --source /kaggle/working/drone_dataset/valid/images \
    --img-size 320 \
    --conf 0.25 \
    --device 0 \
    --name drone-detection
```

Figure 11: AMB82-Mini Drone Detection Demo Model Detection (1)

Figure 12: AMB82-Mini Drone Detection Demo Model Detection (2)

Figure 13: AMB82-Mini Drone Detection Demo Model Detection (3)

8.8 Exporting the Trained Weights

The best checkpoint was copied out of YOLOv7's runs/ directory to a clean, easy-to-find location before conversion:

```
import shutil
from pathlib import Path

source = Path("/kaggle/working/yolov7/runs/train/drone-pretrained-50/weights/best.pt")
destination = Path("/kaggle/working/weights/drone-pretrained-50-best.pt")
destination.parent.mkdir(parents=True, exist_ok=True)
shutil.copy2(source, destination)
```

8.9 Reparameterization for Deployment

YOLOv7's training-time architecture uses re-parameterizable (RepConv-style) branches that need to be fused into a simpler, inference-only architecture before the model can be exported and converted for embedded hardware. Realtek provides reparameterization scripts for exactly this purpose, bundled with the Ameba board package rather than with YOLOv7 itself.

8.9.1 Locating the Reparameterization Scripts

The scripts ship inside the locally installed Ameba board package, under:

```
<Arduino15>\packages\realtek\hardware\AmebaPro2\4.1.0\libraries\NeuralNetwork\src\Yolov7_reparam_scripts
```

> *Since this folder only exists on the local machine (not on Kaggle), the scripts were uploaded to a private Kaggle dataset and imported into the training notebook to run the reparameterization step where a compatible GPU/PyTorch setup was available.*

8.9.2 Compatibility Fix for Newer PyTorch

The bundled reparam script's checkpoint loading assumes PyTorch's older torch.load default. Newer PyTorch versions default to a stricter loading mode that breaks this, and needed a one-line patch:

```
file = Path("/kaggle/working/yolov7/reparam_yolov7-tiny.py")
text = file.read_text()

old = "ckpt = torch.load(args.weights, map_location=device)"
new = "ckpt = torch.load(args.weights, map_location=device, weights_only=False)"

text = text.replace(old, new)
file.write_text(text)
```

⚠ Finding: undocumented PyTorch compatibility fix

Neither the official docs nor the forum thread mention this torch.load patch. Without it, the reparameterization script fails outright on current PyTorch releases because of the changed default for weights_only.

8.9.3 Running the Reparameterization Script

The reparam scripts must be run from inside the yolov7 folder (they're configured to import from it), pointed at the fine-tuned best.pt and a deploy-mode YAML:

```
!mkdir -p /kaggle/working/weights

!python reparam_yolov7-tiny.py \
```

```
    --weights "runs/train/drone-pretrained-50/weights/best.pt" \
    --custom_yaml "yolov7-tiny-deploy.yaml" \
    --output "../weights/drone-pretrained-50-best-reparam.pt"
```

> *General command form, for reference: reparam_yolov7-tiny.py --weights weights/best.pt --custom_yaml custom/yolov7-tiny-deploy.yaml --output best_reparam.pt*

8.10 Online Model Conversion (PyTorch → AMB82-Mini .nb)

The reparametrized .pt file was submitted to Realtek's AmebaPro2 online AI model conversion tool, which compiles the network and returns a ".nb" model file in the board's native format via email, ready to flash.

While **YOLOv7-Tiny (PyTorch)** was used for this project, Realtek's online service supports a variety of popular frameworks and pre-trained architectures for object detection, face recognition, audio analysis, and image classification:

Models	Basic functions
yolov3-tiny, darknet	Object Detection
yolov4-tiny, darknet	Object Detection
yolov7-tiny, darknet	Object Detection
yolov7-tiny, pytorch	Object Detection
scrfd/mobilefacenet	Face Detection & Recognition
yamnet	Sound Classification
CNN Gray/RGB	Image Classification

Table 8: AMB82 Online Model Conversion Models Supported

> *Note: The online tool also accepts up to 10 optional quantization images during submission to assist in model optimization.*

- **Model conversion overview:** amebaiot.com – AmebaPro2 AI Convert Model
- **Applying a converted model:** amebaiot.com – AmebaPro2 Apply AI Model
- **Offline conversion (alternative):** amebaiot.com – Offline AI Model Conversion

The online tool was selected for convenience in this workflow. The offline conversion tool (linked above) serves as a local alternative if you prefer not to upload proprietary weight files to an external web service.

8.11 Deploying to the AMB82-Mini (Arduino IDE)

Prerequisites: Arduino IDE with the Realtek AmebaPro2 board package (v4.1.0) installed, and the AMB82-Mini connected over USB.

> *Recommended: exclude the Arduino15 packages folder from Windows Defender (or your antivirus) before building — e.g. <Users>\<you>\AppData\Local\Arduino15 — large SDK trees like this one can otherwise cause slow or unreliable compiles.*

8.11.1 Where to Place the Model File

Two placements were tested and both worked (no SD card was used — the model was flashed directly):

1. **Directly inside the example folder:** drop the .nb file into the ObjectDetectionLoop example directory itself — `<Arduino15>\packages\realtek\hardware\AmebaPro2\4.1.0\libraries\NeuralNetwork\examples\ObjectDetectionLoop`
2. **A separate sketch folder:** create your own project as a sibling folder under `...\AmebaPro2\4.1.0\libraries\`, and place the .nb file alongside your sketch — e.g. `...\libraries\CUSTOM\ObjectDetectionLoopDrone\`

8.11.2 Adapting the Example Sketch

Starting from the ObjectDetectionLoop example (File → Examples → the NeuralNetwork library), two changes were needed for a custom single-class model:

- **Model file:** point the sketch at your custom ".nb" file instead of the default bundled model
- **Class list:** the stock example is set up for the 80-class COCO model (see ObjectClassList.h); since this model outputs a single "drone" class, the class handling/labels in the sketch were reduced to match

8.11.3 Compile and Upload

1. Tools → Board: select the AmebaPro2 / AMB82-Mini board entry
2. Tools → Port: select the board's COM port
3. Click Upload, and wait for the sketch to compile and flash

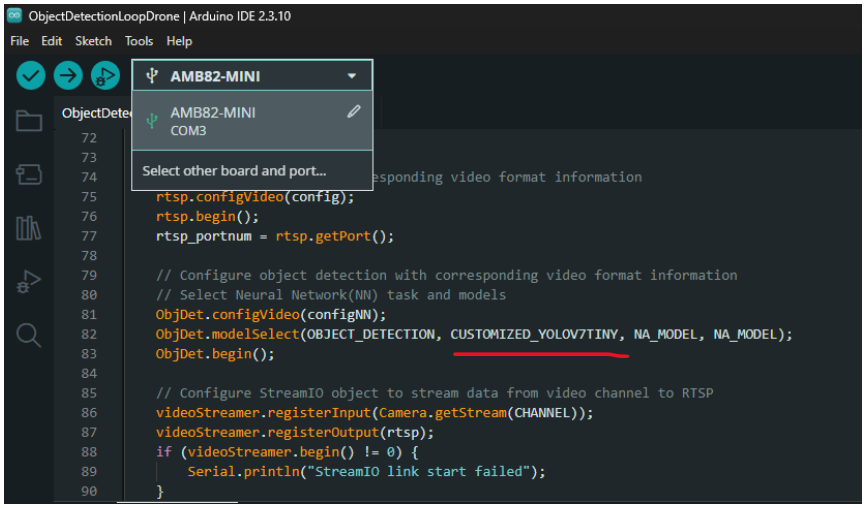


Figure 14: AMB82-Mini Drone Detection Board and Port Seclection

8.12 Viewing the Live Detection Stream in VLC

The AMB82-Mini's neural-network examples stream their camera feed (with detection overlays) over RTSP, which any RTSP-capable player — including VLC — can display.

1. Press the board's Reset button and open the Arduino IDE's Serial Monitor (matching the sketch's baud rate) to watch it join Wi-Fi.
2. Once connected, the Serial Monitor prints the board's IP address and RTSP port (the default RTSP port is 554 unless the sketch overrides it).
3. Install VLC if you don't already have it, and make sure your computer is on the **same network** as the AMB82-Mini.
4. In VLC, go to **Media → Open Network Stream**
5. Enter the stream URL using the IP/port from Serial Monitor, then click **Play**:

```
rtsp://<board-ip-address>:<port>/
# example: rtsp://192.168.1.154:554
```

6. The live camera feed appears in VLC, with a bounding box and label drawn around any detected drone in frame.

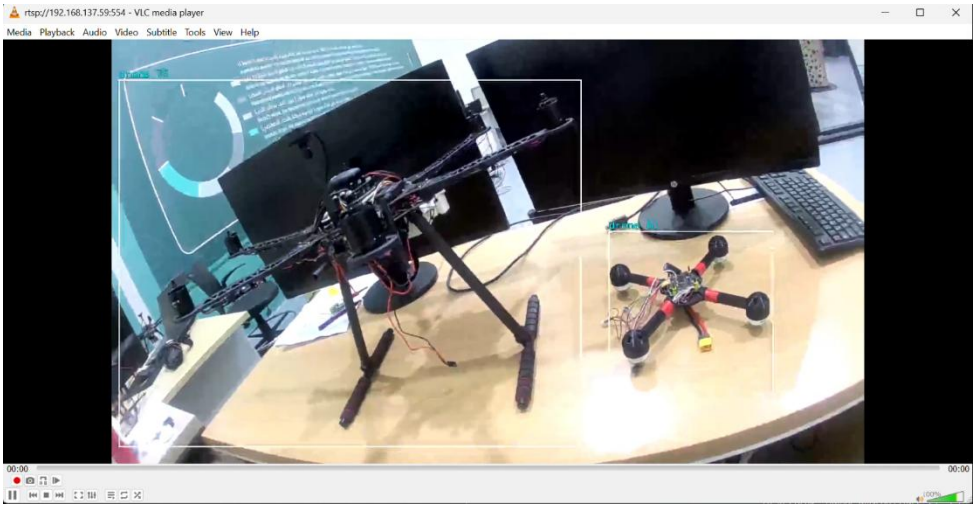


Figure 15: AMB82-Mini Drone Detection Demo Sample

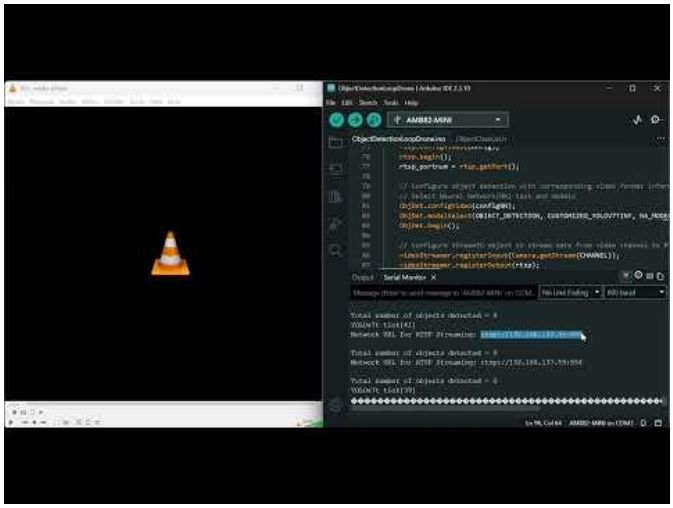


Figure 16: AMB82-Mini Drone Detection Demo Video

8.13 Results

- **mAP@0.5:** The model achieves a mean Average Precision of **0.771** for drone detection at an IoU threshold of 0.5.
- **Peak Metrics:** The optimal F1 score is **0.78** at a confidence threshold of **0.551**. Precision reaches **1.00** at confidence **0.918**, while recall peaks at **0.89** at low confidence thresholds.
- **Live Video Performance:** Video playback during testing was functional but slightly uneven (not perfectly smooth, though acceptable for tracking).
- **Detection Accuracy:** Performance was impacted by dataset size constraints, leading to lower overall accuracy and occasional false positives/negatives that can be resolved with additional data training.

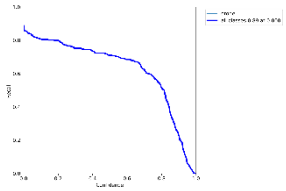


Figure 17: AMB82-Mini Drone Detection R Curve

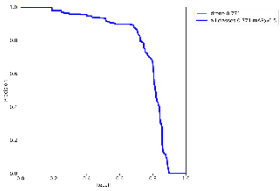


Figure 18: AMB82-Mini Drone Detection PR Curve

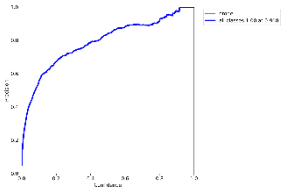


Figure 19: AMB82-Mini Drone Detection P Curve

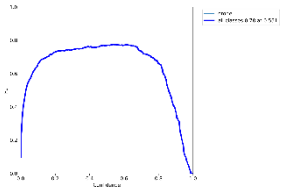


Figure 20: AMB82-Mini Drone Detection F1 Curve

8.14 Findings and Deviations from Documentation

A summary of everything encountered during this project that differed from, or wasn't fully covered by, the official documentation:

- **GPU dependency:** the reparameterization script requires a GPU-compatible PyTorch environment; it failed on a local CPU-only Windows machine but worked on Kaggle
- **PyTorch compatibility:** newer PyTorch's stricter torch.load default (weights_only) broke the bundled reparam script; fixed with a one-line patch not mentioned in the official docs
- **Model file placement:** both tested locations for the .nb model file worked — directly inside the ObjectDetectionLoop example folder, or in a separate sketch folder created under the same libraries directory; SD card integration was not needed
- **Build environment:** excluding the Arduino15 folder from Windows Defender avoided antivirus-related build issues

9 Case Study: Real-Time Drone Detection on the Raspberry Pi Zero 2 W

9.1 Project Overview

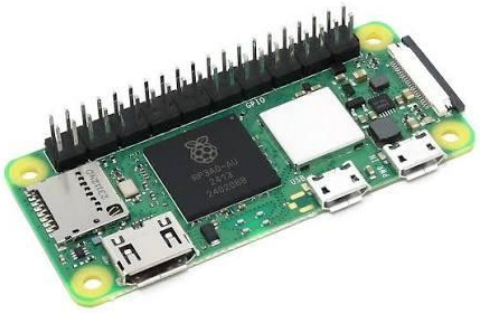


Figure 21: Raspberry Pi Zero 2w

This case study documents the deployment of the same trained YOLOv7 drone-detection model used in the AMB82-Mini case study (Section 8) onto a Raspberry Pi Zero 2 W. The objective was to reproduce the end-to-end edge-inference pipeline — camera capture, preprocessing, on-device inference, postprocessing, and live network streaming — on a lower-cost, general-purpose Linux SBC, and to compare its behaviour and constraints against the AMB82-Mini deployment.
Unlike the AMB82-Mini, which uses Realtek's proprietary NPU toolchain (.nb model format, Arduino IDE flow), the Raspberry Pi deployment uses NCNN, an open-source, CPU-oriented inference framework, with the model exported directly from the same PyTorch weights (best.pt).

9.1.1 Hardware and Software Used

Component	Detail
Board	Raspberry Pi Zero 2 W
Operating System	Ubuntu 22.04.5 LTS (Jammy), aarch64
Kernel	5.15.200
CPU / RAM	4-core CPU; ~414 MB usable RAM; 0 B swap (default)
Camera (attempted)	Raspberry Pi Camera Module 2 — Sony IMX219 sensor, CSI connector
Camera (used)	USB webcam (UVC / V4L2, MJPEG capable)
Inference runtime	NCNN (built from source, Python bindings)
Model	YOLOv7, single class ("drone"), same weights as AMB82-Mini case study
Model files	best.ncnn.param, best.ncnn.bin (converted from best.pt)
Application stack	Python 3.10, OpenCV, NumPy, Flask (MJPEG HTTP stream)
Remote access	SSH via PuTTY (Windows → Raspberry Pi)

Table 9: Real-Time Drone Detection on the Raspberry Pi Zero 2 W Case Study Pipeline

9.2 Model Conversion Pipeline (PyTorch → NCNN)

The same trained checkpoint used for the AMB82-Mini deployment was converted for NCNN rather than for Realtek's NPU. The conversion path was:

Figure 22: Model Conversion Pipeline for Real-Time Drone Detection on the Raspberry Pi Zero 2 W

The .param file describes the network graph (layers, blob names); the .bin file holds the trained weights. Both files are required at runtime and are loaded together by the NCNN Python API on the Pi.

- YOLOv7 export tooling: https://github.com/WongKinYiu/yolov7
- ONNX → NCNN conversion reference: https://github.com/tencent/ncnn/wiki/use-ncnn-with-pytorch-or-onnx

9.3 Raspberry Pi Environment Setup

9.3.1 Remote Access (SSH via PuTTY)

The Pi runs headless Ubuntu, so all setup was performed remotely from Windows using PuTTY over SSH (port 22, host = Pi's LAN IP, login as pi). A successful session presents the pi@pi:~$ prompt.

9.3.2 Base Python / OpenCV Dependencies

Update package lists, then install the core runtime dependencies:

```
sudo apt update

sudo apt install -y \
    python3 \
    python3-pip \
    python3-numpy \
    python3-opencv \
    v4l-utils \
    libopencv-dev
```

9.3.3 Flask (for the network video stream)

```
python3 -m pip install flask

# Verify all core imports
python3 -c "import cv2, numpy, flask; print('All OK')"
```

9.4 Camera Integration

9.4.1 Attempt 1 — CSI Camera (IMX219)

The project initially used the official Raspberry Pi Camera Module 2 (IMX219 sensor) over the CSI ribbon connector. The kernel and V4L2 layer detected the hardware correctly, but the software stack above it could not deliver a usable frame to OpenCV — this path was ultimately abandoned in favour of a USB webcam (Section 9.4.2).

Kernel detection was confirmed with:

```
sudo dmesg | grep -Ei "camera|imx|unicam|csi|libcamera"
```

Key log lines confirmed the sensor, CSI interface, and Unicam driver were all detected (imx219 10-0010, 3f801000.csi, unicam).

Enumerating V4L2 devices:

```
v4l2-ctl --list-devices

# unicam (platform:3f801000.csi):
#   /dev/video0   <- main CSI capture interface
#   /dev/video1   <- sensor metadata device
#   /dev/media3
```

Inspecting sensor capabilities via the sub-device:

```
v4l2-ctl -d /dev/v4l-subdev0 --all

# Sensor: imx219
# Supported resolutions: 3280x2464, 1920x1080, 1640x1232, 640x480
# Supported Bayer formats: SRGGB10_1X10, SRGGB8_1X8
```

A first raw-capture attempt at 640×480 failed (VIDIOC_STREAMON returned -1) because the sensor pad was still configured for its native 3280×2464 resolution, mismatching the requested capture resolution. This was fixed by explicitly setting the sub-device format:

```
sudo v4l2-ctl -d /dev/v4l-subdev0 \
    --set-subdev-fmt=pad=0,width=640,height=480,code=0x300f

# Verify:
v4l2-ctl -d /dev/v4l-subdev0 --get-subdev-fmt
# -> Width/Height: 640/480, Mediabus Code: 0x300f (SRGGB10_1X10)
```

With the sub-device correctly configured, a raw frame was captured successfully (a ~600 KB test.raw, consistent with a 640×480 10-bit Bayer frame), confirming the sensor, CSI link, kernel driver, and V4L2 raw-capture path were all functioning:

```
v4l2-ctl -d /dev/video0 \
    --stream-mmap \
    --stream-count=1 \
    --stream-to=test.raw
```

The blocker was one level up: /dev/video0 exposed raw 10-bit Bayer data rather than a decodable RGB/YUV frame, so cv2.VideoCapture("/dev/video0", cv2.CAP_V4L2) opened the device but every read() failed. The standard fix is the Raspberry Pi ISP/libcamera stack, which converts Bayer data to a usable image — but the Ubuntu 22.04 libcamera package was an outdated 2020 build (cam --version segfaulted), and the expected Pi camera utilities (libcamera-hello, rpicam-hello) were not present.

Building the current Raspberry Pi libcamera stack from source was attempted, but hit two further blockers: Ubuntu's Meson (0.61.2) was older than the version libcamera requires (≥1.0.1), fixed with pip install --user --upgrade meson; and a missing libevent_pthreads dependency, fixed with sudo apt install libevent-dev. The build was then run with ninja -C build -j1 (single-threaded, to respect the Pi's limited RAM), but the overall effort-to-value ratio of continuing down this path was judged too high for the project's goals.

9.4.2 Decision — Switch to a USB Webcam

Given the project's actual requirement — camera → normal RGB/YUV image → preprocessing → YOLOv7/NCNN → postprocessing → network stream — the CSI/IMX219 path added significant, unplanned software-stack work (libcamera/ISP) unrelated to the core edge-AI benchmarking objective. A standard USB (UVC) webcam exposes conventional formats (MJPEG, YUYV, RGB) directly over V4L2, which OpenCV consumes natively via:

```
cap = cv2.VideoCapture("/dev/video0", cv2.CAP_V4L2)
```

This let the project focus on the pipeline that actually matters for benchmarking:

capture → preprocessing → NCNN inference → postprocessing → bounding boxes → network stream, and on measuring per-stage latency and FPS.

9.4.3 Verifying the USB Camera

```
v4l2-ctl --list-formats-ext -d /dev/video0
# Confirms /dev/video0 supports MJPEG @ 640x480 (and other resolutions/formats)

# Ensure the pi user can access the capture device:
groups
```

```
sudo usermod -aG video $USER   # if 'video' is not listed, then reconnect the SSH session
```

Deviation from initial plan
CSI Camera Integration: An IMX219 camera was connected via CSI and successfully detected at the kernel, V4L2 and raw-capture levels (640×480, 10-bit Bayer). However, Ubuntu 22.04's outdated libcamera stack could not convert the raw Bayer data to a usable RGB/YUV frame, and rebuilding a current libcamera from source introduced further dependency issues. Given the Pi's limited resources and the project's real-time benchmarking goal, the CSI path was discontinued in favour of a USB webcam.

9.5 Building NCNN from Source

9.5.1 Why Build from Source

No prebuilt NCNN package was available for this target (Ubuntu 22.04, ARM64, Python 3.10, Raspberry Pi):

```
python3 -c "import ncnn"
# ModuleNotFoundError: No module named 'ncnn'

find /usr /usr/local -iname "*ncnn*"     # no results
ldconfig -p | grep ncnn                  # no results
```

NCNN was therefore compiled from source so that both the native runtime and the Python bindings match the Pi's architecture and Python version.

9.5.2 Install Build Dependencies

```
sudo apt install -y \
    git \
    cmake \
    build-essential \
    python3-dev \
    python3-pip \
    libprotobuf-dev \
    protobuf-compiler \
    libvulkan-dev \
    vulkan-tools
```

9.5.3 Clone and Configure NCNN

```
cd ~
git clone --depth 1 https://github.com/Tencent/ncnn.git
```

```
cd ~/ncnn
mkdir -p build
cd build

cmake .. \
    -DCMAKE_BUILD_TYPE=Release \
    -DNCNN_BUILD_EXAMPLES=OFF \
    -DNCNN_BUILD_TOOLS=ON \
    -DNCNN_PYTHON=ON \
    -DNCNN_VULKAN=OFF
```

NCNN_PYTHON=ON generates the Python bindings (import ncnn); NCNN_VULKAN=OFF targets CPU-only inference, since Vulkan/GPU compute was not used for this deployment. Configuration was confirmed with:

```
grep -E "NCNN_PYTHON|NCNN_VULKAN|NCNN_BUILD_EXAMPLES|NCNN_BUILD_TOOLS" CMakeCache.txt

# NCNN_BUILD_EXAMPLES:BOOL=OFF
# NCNN_BUILD_TOOLS:BOOL=ON
# NCNN_PYTHON:BOOL=ON
# NCNN_VULKAN:BOOL=OFF
```

9.5.4 Resolving the Memory Constraint (Swap)

The Pi Zero 2 W's RAM is not sufficient to compile a large C++ project like NCNN in parallel:

```
free -h
#               total   used   free   available
# Mem:          414Mi   129Mi  240Mi  272Mi
# Swap:            0B

nproc
# 4
```

A 2 GB swap file was created and made persistent across reboots:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

free -h   # Swap should now show ~2.0Gi

# Persist across reboots:
sudo nano /etc/fstab
# add the line:
```

```
# /swapfile none swap sw 0 0
```

9.5.5 Compiling NCNN

Compilation was run single-threaded to avoid exhausting the Pi's limited RAM, relying on the swap file as a safety margin:

```
cd ~/ncnn/build
make -j1
```

Job count	Recommendation on Pi Zero 2 W
make -j1	Safest — one compilation unit at a time; used for this build
make -j2	Possible, with swap enabled; higher OOM risk
make -j4	Not recommended — high risk of running out of memory

Table 10: Job Count Comparison

9.5.6 Verifying the Build and Configuring PYTHONPATH

After compilation, the Python extension module was confirmed to exist:

```
./python/ncnn/ncnn.cpython-310-aarch64-linux-gnu.so
./build/python/ncnn/ncnn.cpython-310-aarch64-linux-gnu.so
```

Despite the .so file existing, a plain import still failed because Python did not know where to find it. Setting PYTHONPATH explicitly confirmed the build was functional:

```
PYTHONPATH=$HOME/ncnn/build/python python3 -c "import ncnn; print('NCNN OK')"
# NCNN OK
```

This was then made permanent by adding the export to the shell profile:

```
echo 'export PYTHONPATH=$HOME/ncnn/build/python:$PYTHONPATH' >> ~/.bashrc
source ~/.bashrc

python3 -c "import ncnn; print('NCNN OK')"
# NCNN OK
```

9.6 Detection Application (detector.py)

With the camera and NCNN runtime both working, a single Python application (detector.py) implements the full pipeline: it loads the NCNN model once at startup, continuously captures frames from the USB camera, runs YOLOv7/NCNN inference and postprocessing on a background thread, overlays results and performance stats, and serves the annotated feed as an MJPEG stream over Flask for viewing in a browser or VLC.

9.6.1 Configuration and Model Loading

Model paths, thresholds, and stream settings are defined as constants at the top of the script, and the NCNN network is loaded once, globally, at import time:

```
MODEL_PARAM = "/home/pi/drone_detection/best.ncnn.param"
MODEL_BIN   = "/home/pi/drone_detection/best.ncnn.bin"

INPUT_BLOB, OUTPUT_BLOB = "in0", "out0"
INPUT_SIZE = 320

CONF_THRESHOLD = 0.25
NMS_THRESHOLD  = 0.45

CLASS_NAMES = ["drone"]

net = ncnn.Net()
net.opt.use_vulkan_compute = False   # CPU inference
net.opt.num_threads = 4

net.load_param(MODEL_PARAM)   # must return 0
net.load_model(MODEL_BIN)     # must return 0
```

9.6.2 Preprocessing — Letterbox Resize

The model expects a fixed 320×320 RGB input. Camera frames (640×480) are letterboxed — resized to fit within 320×320 while preserving aspect ratio, then padded with grey (114,114,114) — before being converted from BGR to RGB and wrapped in an NCNN Mat:

Camera frame (640×480) → letterbox → 320×320 → BGR→RGB → ncnn.Mat.from_pixels() → normalize (÷255)

```
image, scale, pad_x, pad_y = letterbox(frame, INPUT_SIZE)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

mat = ncnn.Mat.from_pixels(
    image, ncnn.Mat.PixelType.PIXEL_RGB, INPUT_SIZE, INPUT_SIZE
)
mat.substract_mean_normalize([0, 0, 0], [1/255.0, 1/255.0, 1/255.0])
```

The scale factor and padding offsets (scale, pad_x, pad_y) are kept so that detection boxes can be mapped back from the 320×320 model space to the original 640×480 camera frame after inference.

9.6.3 NCNN Inference

```
extractor = net.create_extractor()
extractor.set_light_mode(True)

extractor.input(INPUT_BLOB, mat)          # "in0"
ret, output = extractor.extract(OUTPUT_BLOB)   # "out0"

detections = np.array(output)   # shape (6300, 6), dtype float32
```

The .param graph exposes a single output blob (Concat cat_17 → out0) that concatenates YOLOv7's three detection heads — operating on 40×40, 20×20 and 10×10 grids — into one 6300×6 tensor: 6300 candidate predictions, each with 6 values ([cx, cy, w, h, objectness, class_score] as implemented in the postprocessing code below).

9.6.4 Postprocessing — Confidence Filtering, Box Decoding, NMS

Each of the 6300 raw predictions is filtered by confidence (objectness × class_score), converted from centre-coordinates to corner-coordinates, un-letterboxed back to the original 640×480 frame, and clamped to the frame boundary. OpenCV's cv2.dnn.NMSBoxes then removes duplicate/overlapping boxes:

```
confidence = objectness * class_score
if confidence < CONF_THRESHOLD:
    continue

x1, y1 = cx - w/2, cy - h/2
x2, y2 = cx + w/2, cy + h/2

# Undo letterbox back to the original frame
x1, y1 = (x1 - pad_x) / scale, (y1 - pad_y) / scale
x2, y2 = (x2 - pad_x) / scale, (y2 - pad_y) / scale

indices = cv2.dnn.NMSBoxes(boxes, scores, CONF_THRESHOLD, NMS_THRESHOLD)
```

9.6.5 Overlay, FPS Measurement and MJPEG Streaming

Surviving detections are drawn onto the frame (bounding box + label + confidence). Rolling FPS is computed once per second from a frame counter, and both FPS and per-frame inference time (ms) are overlaid on the image. Each processed frame is JPEG-encoded and published to a shared, lock-protected buffer that a Flask route streams as multipart MJPEG:

```
success, jpeg = cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, 80])
```

```
with frame_lock:
    latest_frame = jpeg.tobytes()

@app.route("/stream.mjpg")
def stream():
    return Response(
        generate_stream(),
        mimetype="multipart/x-mixed-replace; boundary=frame"
    )
```

Capture and inference run continuously on a dedicated daemon thread (detection_loop), decoupling frame processing from the Flask HTTP server thread so the stream stays responsive.

9.7 Running the Detector and Viewing the Stream

With NCNN on PYTHONPATH permanently (Section 9.5.6) and the model files in place, the application is started directly:

```
cd /home/pi/drone_detection
python3 detector.py
```

On startup the script loads the NCNN model, opens the USB camera at 640×480 @ 30 FPS (MJPEG), starts the background detection thread, and serves the stream on port 8080:

Access method	URL
Browser (index page)	http://<RASPBERRY_PI_IP>:8080/

Table 11: Real-Time Drone Detection on the Raspberry Pi Zero 2 W Video Stream Access Methods

9.8 Results

The pipeline was verified end-to-end on the Raspberry Pi Zero 2 W: the model loads successfully, the camera streams frames, detections are drawn on-frame, and the annotated feed is viewable live over the network in a browser.

Figure 23: Real-Time Drone Detection Pipeline on the Raspberry Pi Zero 2 W

Metric	Value
Model loaded	Yes — net.load_param() / net.load_model() both returned 0
Model used	YOLOv7-tiny (same weights trained primarily for the AMB82-Mini case study, reused as-is — not retrained or re-optimized for the Pi)
NCNN backend	CPU (Vulkan disabled), 4 threads
Camera	USB webcam, 640×480 @ 30 FPS (MJPEG)
Model input / output	320×320 in0 → 6300×6 out0
Live stream	Verified working — browser (/)
Average FPS (end-to-end)	~1.7 – 1.8 FPS
Average inference time / frame	~500 – 545 ms
Detection accuracy	Good — model correctly localised and classified drones in-frame; observed confidences ranged ~0.38 – 0.56 on the test bench scene (3 objects detected simultaneously)

Table 12: Real-Time Drone Detection on the Raspberry Pi Zero 2 W Results

Live stream — browser view (http://<PI_IP>:8080/):

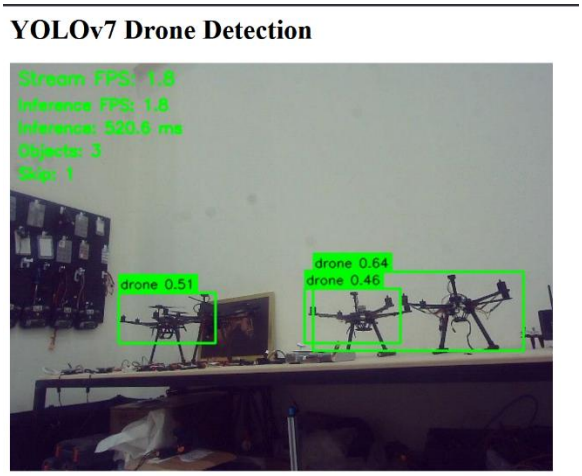


Figure 24: Live MJPEG detection stream - Raspberry Pi Zero 2 W (Yolov7-tiny)

Note on the model

The .pt weights used here are a YOLOv7-tiny model trained primarily for the AMB82-Mini case study (Section 8) and converted/deployed on the Pi as-is, with no Pi-specific retraining, pruning, or quantization. Detection quality (accuracy/confidence) was good, but end-to-end throughput was low — this is expected for a tiny-but-still-full-precision YOLO model run purely on Pi Zero 2 W CPU, and is analysed in Section 9.9 below.

9.9 Performance Analysis and Optimization Recommendations

At ~1.7–1.8 FPS and ~500+ ms inference time, the current deployment is far from real-time (typically ≥10–15 FPS is desired for a responsive detection stream). Accuracy itself was not the bottleneck — the model correctly detected and localised drones with reasonable confidence — so the priority for improvement is throughput/latency, not retraining for correctness. The main contributing factors and corresponding options are summarised below.

9.9.1 Why Performance Is Low

- Hardware ceiling: the Pi Zero 2 W has a modest quad-core Cortex-A53 CPU with no NPU/GPU acceleration path exercised here (NCNN_VULKAN was disabled) and only ~512 MB RAM, so all inference work is done on general-purpose CPU cores.
- 320×320 input on CPU: even at a relatively small 320×320 input, three detection heads (40×40, 20×20, 10×10 → 6300 total predictions) is a meaningful amount of compute per frame for this CPU class.

9.9.2 Recommended Improvements

Option	Expected impact
Smaller input resolution (e.g. 256×256 or 192×192 instead of 320×320)	Roughly quadratic reduction in compute with resolution; directly cuts inference time, at some cost to small-object recall
Switch to a smaller/faster architecture (YOLOv5n, YOLOv8n, NanoDet, or YOLO-Fastest) trained/exported for this deployment	These nano-class models are purpose-built for CPU/edge SBCs and are typically several times faster than YOLOv7-tiny at similar accuracy for single-class detection
Re-train or fine-tune specifically for the Pi target (rather than reusing AMB82-Mini weights as-is)	Lets the model/input size be chosen for this hardware's actual compute budget instead of inheriting NPU-oriented choices
Frame-skipping / lower inference rate	Run detection every Nth frame and interpolate/hold the last box between detections — trades detection latency for perceived smoothness, no model change needed
Reduce camera capture resolution (e.g. 480×320) before letterboxing	Cuts preprocessing (resize/color-convert) cost per frame; minor but free win

Table 13: Real-Time Drone Detection on the Raspberry Pi Zero 2 W Recommendations

Given the model already showed good detection accuracy, the recommended next step is benchmarking a purpose-built nano model (YOLOv8n/YOLOv5n) at a reduced input size. This also gives a natural point of comparison against the AMB82-Mini case study: NPU-accelerated inference (AMB82-Mini) vs. optimized CPU inference (Raspberry Pi Zero 2 W).

9.10 Findings and Deviations from the Original Plan

- CSI camera abandoned: the IMX219/CSI camera was fully functional down to raw V4L2 capture, but Ubuntu 22.04's outdated libcamera stack could not produce a decodable RGB/YUV frame, and rebuilding libcamera from source introduced further dependency issues (Meson version, libevent). A USB webcam was substituted, which OpenCV consumes natively.
- No prebuilt NCNN package existed for this target; NCNN had to be compiled from source, which required creating a 2 GB swap file and building single-threaded (make -j1) due to the Pi's ~414 MB RAM.
- The compiled NCNN Python binding was not automatically importable — PYTHONPATH had to be set explicitly (and then persisted in ~/.bashrc) even though the .so file existed.
- The model and class list are single-class ("drone"), matching the AMB82-Mini case study's trained weights.
- Performance vs. accuracy split: detection accuracy was good (correct localisation/classification at reasonable confidence), but end-to-end throughput was poor (~1.7–2.0 FPS, ~500+ ms/frame) because a YOLOv7-tiny model trained for the AMB82-Mini's NPU was reused unmodified on Pi CPU-only inference. See Section 9.9 for concrete optimization options (smaller/nano architectures, reduced input size).

9.11 Performance Improvement Iteration: Fine-Tuning YOLO11n

Following the optimization roadmap in Section 9.9, this iteration acts on two of its concrete recommendations directly: moving to a purpose-built “nano” architecture instead of reusing the AMB82-oriented YOLOv7-tiny weights, adding frame-skipping to reduce how often the (comparatively expensive) inference step runs and reduce camera capture resolution. The model was retrained from scratch on the project's own dataset via transfer learning, and the Raspberry Pi application was updated accordingly.

> **Note**
> The frame-skipping and reduce camera capture resolution improvements weren’t useful as in real-time application with high speed UAVs frame-skipping is impractical and reducing camera capture resolution redeuces camera quality and accuracy so these improvements are impractical and all ahead measured performance is given without frame-skipping (inferenced every second) and used original camera capture resolution.

9.11.1 Motivation

- Section 9.9 identified two root causes of poor throughput: an NPU-sized model (YOLOv7-tiny, reused unmodified from the AMB82-Mini case study) running at full FP32 precision on CPU-only hardware.
- Recommended fixes included switching to a nano-class architecture (YOLOv5n/YOLOv8n-class) and adding frame-skipping — this iteration implements both, using Ultralytics YOLO11n as the nano architecture.

9.11.2 Training Setup

The same source dataset used for the original YOLOv7-tiny training was reused directly, so results are comparable on a like-for-like data basis:

Item	Detail
Dataset	muki2003/yolo-drone-detection-dataset (Kaggle) — same dataset as the original training
Images	1,012 training images / 347 validation images, single class ("drone")
Base model	yolo11n.pt — Ultralytics' pretrained (COCO) YOLO11-nano checkpoint, fine-tuned via transfer learning
Training platform	Kaggle notebook, NVIDIA Tesla T4 GPU, Ultralytics 8.4.128 / PyTorch 2.10
Key hyperparameters	imgsz=320, epochs=100 (max), batch=16, patience=20 (early stopping), optimizer=auto, seed=42, pretrained=True

Table 14: Real-Time Drone Detection on the Raspberry Pi Zero 2 W With Yolo11n Training Setup

Training stopped early at epoch 85 (no improvement for 20 epochs; best weights saved from epoch 65), completing in approximately 15.5 minutes on the T4 GPU.

9.11.3 Training Results

Metric	Value
Precision	0.896 (89.6%)
Recall	0.816 (81.6%)
mAP@50	0.896 (89.6%)
mAP@50–95	0.539 (53.9%)
Parameters	≈ 2.59M (182 layers during training; 101 fused layers at inference)

Table 15: Real-Time Drone Detection on the Raspberry Pi Zero 2 W With Yolo11n Training Results

9.11.4 Model Export (PyTorch → NCNN)

Export used Ultralytics' built-in NCNN export path (model.export(format="ncnn")), which drives the same PNNX/NCNN conversion tooling referenced in Section 9.2, but invoked directly from the training framework rather than as a separate manual step.

Artifact	.param size	.bin size
NCNN export — FP32	0.02 MB	9.88 MB

Artifact	.param size	.bin size
NCNN export — FP16 (quantize=16)	0.02 MB	4.96 MB (≈ half of FP32)

Table 16: Real-Time Drone Detection on the Raspberry Pi Zero 2 W With Yolo11n Model Export

The exported output tensor has shape (1, 5, 2100) — 5 values (cx, cy, w, h, confidence) per candidate, across 2,100 candidates. This differs from the YOLOv7-tiny export used previously (6 values: cx, cy, w, h, objectness, class_score) — YOLO11's anchor-free, decoupled head collapses objectness and class score into a single confidence value for a single-class model, so the Raspberry Pi postprocessing code had to be rewritten for this shape (see 9.11.5).

> **FP16 export available but not yet used**
>
> *An FP16-quantized NCNN export was also produced (≈ half the .bin size of the FP32 version) but yolo11n_detector.py currently loads the FP32 weights (model.ncnn.param/.bin) by default. Switching to the FP16 export is a low-effort follow-on optimization worth benchmarking on the Pi.*

9.11.5 Updated Detection Application (yolo11n_detector.py)

The application keeps the same overall architecture as Section 9.6 (camera → preprocess → NCNN → NMS → Flask/MJPEG) with several targeted changes:

- Separate FPS tracking: the app now reports Stream FPS (camera/display loop rate) and Inference FPS (actual YOLO run rate) independently, rather than a single combined FPS figure, giving a more honest picture of where time is spent.
- Updated postprocessing for YOLO11's (5, 2100) output: reads confidence directly from row 4, with no objectness × class_score multiplication step (see 9.11.4).
- Tightened thresholds: CONF_THRESHOLD raised from 0.25 → 0.45 and NMS_THRESHOLD from 0.45 → 0.55, to compensate for the higher detection density typical of anchor-free heads.
- On-screen overlay extended with a "Skip: N" readout alongside Stream FPS, Inference FPS, Inference (ms), and Objects, so the active frame-skip setting is always visible on the stream.
- Camera device path updated to /dev/video1 (device enumeration differed on this run of the Pi).

Key excerpt — dual FPS tracking and frame-skip logic:

```
if frame_count % FRAME_SKIP == 0:
    # Run YOLO inference this frame
    output, scale, pad_x, pad_y = run_inference(frame)
    detections = postprocess(frame, output, scale, pad_x, pad_y)
    last_detections = detections
    processed_frames += 1
else:
    # Reuse the previous detection — no inference this frame
```

```
    detections = last_detections

# Once per second:
stream_fps = displayed_frames / elapsed       # camera/display loop rate
inference_fps = processed_frames / elapsed    # actual YOLO run rate
```

9.11.6 On-Device Results (Raspberry Pi Zero 2 W)

Deployed via the same NCNN Python pipeline as Sections 9.6–9.7, using model.ncnn.param/model.ncnn.bin in place of best.ncnn.*.

Metric	Value
Inference FPS (actual YOLO rate)	~2.6 – 2.8 FPS
Inference time / processed frame	~300 – 350 ms
Frame skip setting used	1 (every frame)

Table 17: Real-Time Drone Detection on the Raspberry Pi Zero 2 W With Yolo11n On-Device Results

Figure 25: Live MJPEG detection stream - Raspberry Pi Zero 2 W (Yolo11n)

9.11.7 Summary: YOLOv7-tiny vs. YOLO11n on the Raspberry Pi Zero 2 W

Aspect	YOLOv7-tiny (Sections 9.1–9.10)	YOLO11n (this iteration)
Origin	Trained for the AMB82-Mini NPU, reused as-is on the Pi	Fine-tuned specifically for this task via transfer learning (Kaggle T4 GPU)
Measured Pi FPS	≈ 1.7–1.8	~2.6 – 2.8 FPS

Aspect	YOLOv7-tiny (Sections 9.1–9.10)	YOLO11n (this iteration)
Measured Pi inference time	≈ 500–545 ms	~300 – 350 ms

Table 18: Real-Time Drone Detection on the Raspberry Pi Zero 2 W - Yolov7-Tiny VS Yolo11n

With the on-Pi performance numbers now recorded, fine-tuning YOLO11n specifically for this deployment—rather than reusing a model targeted at a different architecture—reduced inference latency from ~500–545 ms down to ~300–350 ms (~2.6–2.8 FPS). These optimizations demonstrate how leveraging a modern, lower-parameter architecture directly translates to performance gains on resource-constrained hardware like the Pi Zero 2 W.

10 Benchmarking and Performance Comparison: AMB82-Mini vs. Raspberry Pi Zero 2 W (YOLOv7-tiny vs. YOLO11n)

This section compares the edge-deployment work completed in this project across three configurations: the AMB82-Mini running YOLOv7-tiny on its NPU (Section 8), the Raspberry Pi Zero 2 W running the same YOLOv7-tiny weights on CPU (Section 9.1–9.10), and the Raspberry Pi Zero 2 W running a separately fine-tuned YOLO11n model with frame-skipping (Section 9.11). The first two isolate the effect of hardware (NPU vs. CPU) on identical weights; the second two isolate the effect of a model/optimization change on identical hardware.

10.1 Overview and Methodology

Two comparisons are layered in this section:

- Hardware comparison (same model): AMB82-Mini (NPU) vs. Raspberry Pi Zero 2 W (CPU), both running the identical YOLOv7-tiny weights — isolates the effect of dedicated NPU acceleration vs. general-purpose CPU.
- Model/optimization comparison (same hardware): Raspberry Pi Zero 2 W running YOLOv7-tiny (reused, untuned) vs. the same Pi running a separately fine-tuned YOLO11n — isolates the effect of choosing a purpose-built nano architecture and adding an application-level optimization.

None of the three configurations were benchmarked under an identical, controlled test harness — timing and accuracy were captured using whatever instrumentation was actually available for each (see the Speed and Accuracy sections below), so this comparison should be read as directionally useful rather than a formal, apples-to-apples benchmark. Caveats are collected in the Limitations section at the end.

10.2 Detection Accuracy

On both platforms, the model showed decent detection accuracy at close range: drones placed within the camera's near field were correctly localised and classified, with reasonable confidence scores on both the AMB82-Mini and the Raspberry Pi Zero 2 W.

Important caveat
"Decent accuracy at close range" is a bench-top, visual observation on both platforms, not a quantitative metric (e.g., mAP/precision/recall) computed against a labelled validation set. It also does not necessarily indicate the models will behave the same way in practical field conditions — greater distance, altitude, motion blur, lighting variation, and a moving (rather than static) target could all affect the two platforms differently, especially given their very different compute budgets.

10.3 Inference Speed and FPS Comparison

10.3.1 Raspberry Pi Zero 2 W — YOLOv7-tiny (Measured)

The original Raspberry Pi deployment (detector.py, Section 9.6–9.7) instruments its own timing directly: FPS is computed once per second from a frame counter, and inference time is measured with time.time() around the NCNN extractor.extract() call. Both figures are overlaid live on the video stream and were confirmed visually during a live run:

Metric	Measured Value
Average FPS (end-to-end)	~1.7 – 1.8 FPS
Average inference time / frame	~500 – 545 ms
Measurement method	Application-level timer (time.time()) around the NCNN inference call, plus a rolling frame counter for FPS

Table 19: Raspberry Pi Zero 2W (yolov7-tiny) Measured Performance

10.3.2 Raspberry Pi Zero 2 W — YOLO11n (Measured)

The updated application (yolo11n_detector.py, Section 9.11.5) extends the same measurement approach:

Metric	Measured Value
Average FPS (end-to-end)	~2.6 – 2.8 FPS
Average inference time / frame	~300 – 350 ms
Measurement method	Application-level timer (time.time()) around the NCNN inference call, plus a rolling frame counter for FPS

Table 20: Raspberry Pi Zero 2W (yolov11n) Measured Performance

10.3.3 AMB82-Mini — Approximate Performance (NPU Asynchronous Inference)

Unlike the Pi deployment, the AMB82-Mini's example NN application runs inference asynchronously on the NPU: the Arduino-side application code hands a frame to the NPU and continues, rather than blocking on a synchronous call that could be timed directly with a simple stopwatch pattern (e.g. millis() before/after). No straightforward workaround was found to measure wall-clock inference time from application code the same way it was measured on the Pi.
However, the Realtek NN runtime itself emits internal debug logging over serial while a model is running — periodic tick[N] counters and, every so often, a computed FPS line. This is a documented, known behaviour of the SDK (it is common enough that Realtek provides an FAQ and a Tools → menu option specifically to disable it), and a real forum thread from another developer using the exact same AMB82-Mini YOLOv7-tiny example shows the identical log pattern.

A representative excerpt from the AMB82-Mini's own console output:

```
YOLOv7t tick[39]
YOLOv7t tick[41]
YOLOv7t FPS = 7.06
YOLOv7t tick[40]
YOLOv7t tick[44]
```

Because this FPS figure is computed and printed by the neural-network runtime itself — i.e. it reflects the NPU's own inference loop timing rather than an external application-level stopwatch — it is taken as a reasonable approximation of the AMB82-Mini's real-world inference throughput for this model, even though it was not captured via a custom timing harness.

Metric	Approximate Value
Approx. FPS (from NN runtime log)	~7.06 FPS
Approx. inference time / frame (derived: 1 ÷ FPS)	~142 ms (derived, not directly logged)
Measurement method	Internal NN runtime debug logging (tick[N] / FPS lines) — no application-level timing hook was available, since inference runs asynchronously on the NPU

Table 21: AMB82-Mini Approximate Performance

10.3.4 Side-by-Side Comparison

Metric	AMB82-Mini (YOLOv7-tiny, NPU)	Raspberry Pi (YOLOv7-tiny, CPU)	Raspberry Pi (YOLO11n, CPU)
FPS	~7.06 (approx.)	~1.7–1.8 (measured)	~2.6–2.8 (measured)
Inference time	~142 ms (derived)	~500–545 ms (measured)	~300–350 ms (measured)
Measurement method	Indirect (NN log)	Direct (app timer)	Direct (app timer, once measured)

Table 22: Side-by-Side Performance Comparison - Raspberry Pi Zero 2W (yolov7-tiny and yolo11n) vs AMB82-Mini

Even accounting for the fact that the two figures were captured with different methods, the gap is large enough (multiple times higher FPS on the AMB82-Mini) that the underlying conclusion — the AMB82-Mini runs this model meaningfully faster than the Raspberry Pi Zero 2 W — is very unlikely to be a measurement artefact alone.

10.4 Why the AMB82-Mini Outperforms the Raspberry Pi Zero 2 W

Two separate effects are stacked across these three configurations: a hardware effect (NPU vs. CPU) and a model/optimization effect (NPU-sized model reused as-is vs. a nano model fine-tuned.

10.4.1 Model Size vs. Target Hardware

YOLOv7-tiny, as deployed on the Pi, is the same checkpoint originally trained and sized for the AMB82-Mini's NPU — “tiny” relative to full YOLOv7. The same model that runs comfortably on a purpose-built NPU becomes a stretch for four general-purpose Cortex-A53 cores also handling the OS, camera capture, and network streaming. YOLO11n, in contrast, is a nano-class architecture (≈ 2.59M parameters) fine-tuned specifically for this dataset and this deployment via transfer learning — it was chosen and trained with CPU-class edge hardware in mind, not inherited from a different, accelerator-equipped target.

10.4.2 Dedicated NPU vs. General-Purpose CPU

The AMB82-Mini includes a separate, dedicated Neural Processing Unit — fixed-function silicon built specifically to execute the matrix-multiply/convolution operations neural networks are made of, in parallel, alongside (not instead of) the main CPU. The Raspberry Pi Zero 2 W has no such block: its Cortex-A53 CPU cores are general-purpose, executing convolution as a long sequence of scalar/vector instructions, one layer at a time, with no silicon specialised for this kind of workload. This is the single biggest architectural reason for the gap — it mirrors exactly the NPU-vs-CPU distinction called out for the AMB82-Mini deployment in Section 8.

10.4.3 Toolchain and Runtime Optimization

The AMB82-Mini's deployment path (Section 8: reparameterization → online conversion → .nb → Arduino IDE) is Realtek's own, purpose-built pipeline for getting a model onto its NPU — the model, runtime, and hardware are co-designed and officially supported together. The Raspberry Pi deployment, by contrast, reuses the same AMB82-oriented weights unmodified through a generic, CPU-oriented framework (NCNN) at full FP32 precision, with no Pi-specific pruning, quantization, or architecture changes (Section 9.9). In other words, part of the gap is not just "NPU vs. CPU" in the abstract — it is a well-optimized, hardware-matched pipeline vs. an off-the-shelf model run as-is on hardware it was never tuned for.

10.5 Consolidated Comparison Table

Aspect	AMB82-Mini (YOLOv7-tiny)	Raspberry Pi (YOLOv7-tiny)	Raspberry Pi (YOLO11n)
Compute	Dedicated NPU (RTL8735B)	CPU-only (Cortex-A53)	CPU-only (Cortex-A53)
Model format	Realtek .nb (NPU-optimized)	NCNN .param/.bin	NCNN .param/.bin (FP16 exported)
Model tuned for this deployment?	Yes — native NPU export	No — AMB82 weights reused as-is	Yes — fine-tuned via transfer learning
Accuracy	Good (visual check)	Good (visual check)	Good (visual check)
FPS	~7.06 (approx.)	~1.7–1.8 (measured)	~2.6–2.8 (measured)
Inference time	~142 ms (derived)	~500–545 ms (measured)	~300–350 ms (measured)
Primary bottleneck	—	CPU-only inference of an NPU-sized model	CPU-only inference

Table 23: Comparison - Raspberry Pi Zero 2W (Yolov7-tiny and Yolo11n) vs AMB82-Mini

10.6 Limitations and Caveats

- Not a controlled benchmark: the three configurations were captured with different tools, at different times, and were not run against the same fixed test video/scene, distance, or lighting.
- AMB82-Mini FPS is indirect: it comes from the NN runtime's own internal debug log rather than an application-level timer, and it is unclear whether it includes the full per-frame pipeline (e.g. camera capture, any post-processing/drawing) the same way the Pi's figure does — it may only reflect the NPU inference step itself.
- The ~142 ms AMB82-Mini inference time is a derived value (1 ÷ FPS), not a value the runtime logs directly, and should be treated as an approximation.
- "Decent accuracy at close range" is a qualitative, visual observation on both platforms, not a measured metric (e.g., mAP) on a common labelled test set — confidence thresholds and NMS settings were not confirmed to be identical between the two implementations.
- Field/practical performance may differ from this bench-top comparison: real deployments involve varying distance, altitude, motion, and lighting that were not systematically varied here on either platform.

11 Drone Fundamentals and Practical Lab Experience

This section summarizes background research into UAV/drone technology and controllers, and practical experience gained during a lab visit. It provides context for the detection-focused case studies in Sections 8 and 9 by covering how drones are built, flown, and controlled, and how they can be detected or tracked outside of the vision-based approach used in this project.

11.1 UAV Overview

A UAV (Unmanned Aerial Vehicle) is an aircraft that operates without an onboard pilot, either remotely controlled or flown autonomously via programmed flight systems. A complete UAV is made up of several interconnected subsystems:

- Airframe — the physical structure.
- Propulsion — motors, ESCs, and propellers that generate thrust.
- Flight control — stabilizes and controls the aircraft.
- Navigation/positioning — GPS and related sensors.
- Communication — data link with the ground/operator.
- Power — batteries and power management.
- Payload — mission equipment (cameras, sensors, etc.).

11.1.1 Types of UAVs

Type	Characteristics	Typical Use
Fixed-wing	Airplane-like; needs forward motion; longer endurance, speed, and range (~1–2 hr, 60–120 km/h); cannot hover	Surveying, agriculture, mapping, long-distance monitoring
Multirotor	Multiple propellers; stable hovering; shorter flight time (~15–30 min, 20–50 km/h)	Photography, inspection, search & rescue, delivery
Single-rotor	Helicopter-like; higher payload and longer flight than multirotor	Heavy-lift applications
VTOL / Hybrid	Vertical takeoff/landing combined with fixed-wing flight in the air	Long-range mapping, inspection, surveillance

Table 24: Types of UAVs

The choice of platform matters for a vision-based detection project since a target drone's apparent size, speed, altitude, and motion pattern vary considerably between these types.

Figure 26: Fixed-wing UAV

Figure 27: Multirotor UAV

Figure 28: Single-rotor UAV

Figure 29: VTOL UAV

11.1.2 Core Components

- Motors — usually brushless DC; key specs: KV rating, max thrust, efficiency, weight, voltage.
- ESC (Electronic Speed Controller) — regulates motor speed based on flight-controller commands.
- Propellers — convert rotation to thrust; key specs: diameter, pitch, material, blade count.
- Flight controller — the "brain"; fuses gyroscope, accelerometer, GPS, barometer and magnetometer data for stabilization, waypoint navigation, return-to-home, and auto-landing.
- Battery — typically Li-Po for high energy density; key specs: voltage, capacity, discharge rate, cell configuration.

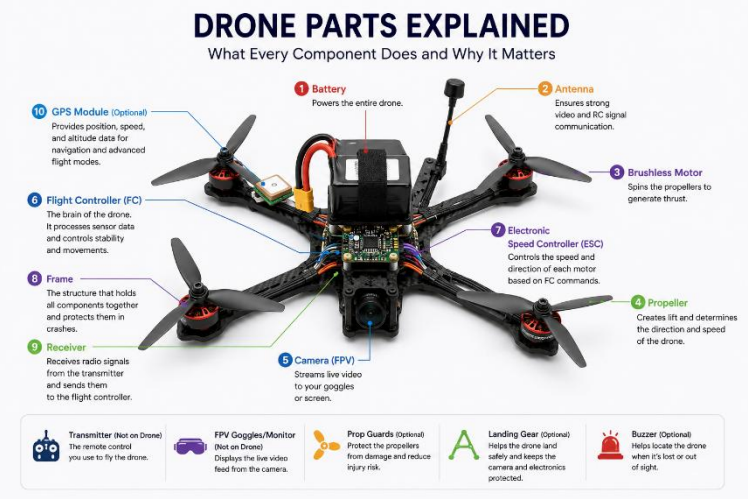


Figure 30: Drone Components

11.1.3 Payloads and Industrial Applications

Payload — high-resolution cameras, LiDAR, thermal/multispectral sensors, spraying rigs, or delivery equipment — determines the UAV's mission. Common industrial applications include agriculture (crop monitoring, spraying), infrastructure inspection (power lines, bridges, pipelines), mapping/surveying, emergency response, construction, delivery, environmental monitoring, real estate/film, mining, and oil & gas inspection.

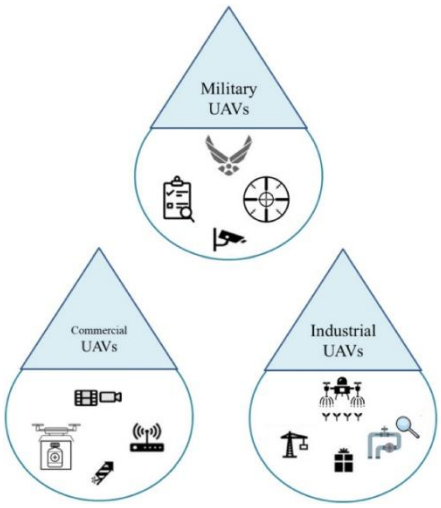


Figure 31: Types of UAV based on applications

11.1.4 Advantages and Emerging Trends

UAVs offer faster data collection, lower operating cost than manned aircraft, safer inspection of hazardous environments, and precise GPS/sensor-based data capture. Emerging trends include AI-powered drones, drone swarms, hybrid VTOL designs, improved batteries, 5G connectivity, night vision, solar power, and underwater-capable drones.

11.2 Drone Controllers and Communication

The controller is the communication link between pilot and UAV: it converts joystick/button inputs into digital/RF signals that the drone's onboard receiver interprets to drive motors and other systems.

11.2.1 Communication Technologies

Technology	Characteristics
2.4 GHz radio	Good range and penetration; can suffer Wi-Fi interference
5.8 GHz radio	More responsive, less interference; shorter range
Wi-Fi	Convenient; typically shorter range than dedicated radio
4G/5G	Long-distance control where cellular coverage exists

Table 25: UAV Communication Technologies

11.2.2 Controller Components and Types

- Joysticks — left: throttle (altitude) and yaw (rotation); right: pitch (forward/back) and roll (left/right).
- Antennas — transmit/receive signals; better antennas improve range and reliability.

- Display/app — live video, maps, GPS position, battery level, telemetry.
- Programmable buttons — return-to-home, camera control, autonomous modes.
- Controller types — standard RC, smartphone/tablet, smart controller (built-in display), and FPV controllers (used with FPV goggles).

Figure 32: Reference UAV Controller

11.2.3 Advanced Features, AI and Autonomy

Modern controllers can add GPS, return-to-home, waypoint navigation, pre-programmed flight paths, and real-time telemetry (altitude, distance, speed, battery). AI increasingly supports autonomous flight, intelligent obstacle avoidance, and predictive maintenance, following a simple loop:

Figure 33: AI - Flight Control Loop

> **Relevance to Edge AI**
> *Processing sensor data locally on the UAV (rather than continuously transmitting it to a remote server) is a direct example of Edge AI in practice, and is the same principle applied by the detection pipelines in Sections 8 and 9.*

11.3 Control Signal Path and RF-Based Counter-Drone Detection

11.3.1 Control Signal Pipeline

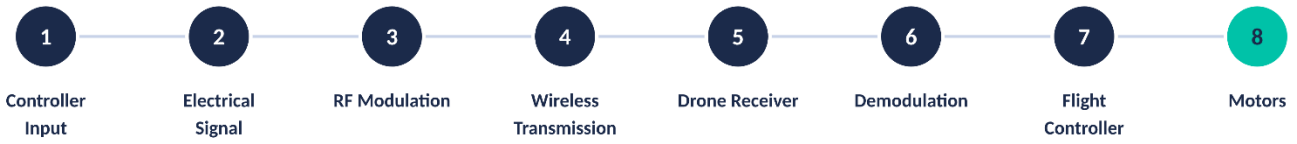


Figure 34: Control Signal Pipeline

Communication is bidirectional: alongside control commands, the UAV also sends telemetry back to the operator (altitude, speed, battery, position), commonly over the 2.4 GHz or 5.8 GHz ISM bands, which trade off range against data-transfer capability.

11.3.2 RF-Based Detection, Tracking, and Mitigation

Counter-drone systems aim to detect, track, and mitigate unauthorised UAVs, commonly via RF sensing of the communication link between drone and controller rather than a camera:

Capability	Purpose
RF Detection	Determine whether drone-related RF signals are present
RF Tracking	Monitor/locate the drone using its RF transmissions (location, altitude, transmission characteristics)
RF Jamming/Mitigation	Disrupt the communication/control link

Table 26: RF-Based Detection, Tracking, and Mitigation

Some advanced systems use software-defined radio (SDR) to adapt to different signal protocols/threats.

11.3.3 Relevance to This Project

> **Vision-based vs. RF-based detection**
> *RF-based detection senses a drone's radio communication with its controller, whereas this project's YOLOv7/NCNN pipeline (Sections 8–9) is vision-based, detecting the drone's physical appearance in camera frames. The two approaches sense fundamentally different signals and are complementary rather than interchangeable — a full counter-drone system would typically combine both.*

11.4 Lab Visit — Practical Experience

In addition to the desk research above, a lab visit provided hands-on exposure to real UAV hardware, ground-control software, and assembly/maintenance procedures.

11.4.1 Hands-on Activities

- Installed propeller blades on a drone.
- Performed wire management on a drone.
- Handled equipment and provided support during drone testing.
- Assisted with experimentation.
- Disassembled a drone for battery charging.

11.4.2 Mission Planner and Telemetry

Mission Planner is a Ground Control Station (GCS) used with ArduPilot-based UAVs for configuration, mission/waypoint planning, monitoring, firmware management, and flight-log analysis. Telemetry carries UAV status (position, altitude, speed, battery) between the UAV and the ground station:

Figure 35: Telemetry Piepline Loop

Figure 36: Mission Planner Interface

11.4.3 Drone Calibration (IMU)

IMU calibration ensures accurate accelerometer and gyroscope readings for the flight controller; incorrect calibration can cause drift, instability, and inaccurate orientation. Calibration is normally performed on a stable, level surface before flight, and is combined with GPS and barometer data through sensor fusion for better flight-state estimation.

11.4.4 Core Components Observed On-Site

- GPS/GNSS — positioning and navigation.
- Flight control board — processes sensor data and drives the motors.
- Battery — powers propulsion, control, communication, and payload.
- Transmitter/receiver — operator ↔ UAV communication link.
- Camera/sensors — environmental data, and a potential input for AI-based detection.

11.4.5 Drone Manufacturing and Construction

UAV manufacturing spans airframe design, component integration, assembly, calibration, and testing. Carbon fiber is commonly used for frames due to its high strength-to-weight ratio, since overall weight directly affects power consumption, payload capacity, and flight endurance. At minimum, a basic quadcopter consists of a frame, motors, propellers, ESCs, a flight controller, a battery, and a receiver.

References

Edge AI, Pipeline & Deployment

Edge Impulse – What Is Edge AI?

Edge Impulse – Edge AI Lifecycle

Hardware

Expressif_esp32_datasheet
mlsysbook.ai – Raspberry Pi Kit Guide

Etteplan – Edge AI Hardware Selection

dev.to – Selecting Embedded LLM/AI Hardware: Points to Consider

Seeed Studio – AMB82-Mini Product Page

Model Characteristics, Evaluation & Deployment Performance

IBM – Model Parameters

How AI Works – Model Size

GeeksforGeeks – Model Complexity & Overfitting

IBM – Confusion Matrix

GeeksforGeeks – Regression Metrics

IBM – Model Selection

Salesforce – What Is an AI Model?

USNI – What Makes a Machine Learning Model Useful

Oracle Blogs – Choosing the Right Type of Model

SmartDev – AI Model Training

TrueFoundry – What Is AI Model Deployment

Nebius – AI Model Performance Metrics

Promwad – Edge AI Model Deployment

MeetHayat – What Is Edge AI Deployment: A Practical Guide

MDPI Electronics – Edge AI Model Deployment Study

Datasets

Hugging Face – Open Datasets and Tools Overview

Xenoss – AI & Data Glossary: Custom Dataset

Macgence – Model-Ready Datasets

Medium (S. Bayar) – Training a Model on a Custom Dataset: Step-by-Step Guide

IBM – Pretrained Models

Medium – 30 Websites for Finding Datasets

Optimization & Quantization

Hugging Face / PrunaAI – Introduction to AI Model Optimization Techniques

Edge Impulse – 7 Tips for Optimizing AI Models for Tiny Devices

NVIDIA Developer – Top 5 AI Model Optimization Techniques

Medium (E. Ejembi) – Optimizing AI Models for Real-World Performance

Optimizing Edge AI for Real-Time Data Processing in IoT Devices: Challenges and Solutions

How to Optimize AI Model Performance for Edge Deployment in IoT and Manufacturing Applications

How to Choose the Right Edge Device for VisionAI - Blog

How to Choose the Right Edge Device for VisionAI – Youtube Video

Optimizing Your AI Model for the Edge

Google – LiteRT Quantization Guide

NVIDIA Developer – Model Quantization: Concepts, Methods, and Why It Matters

Hugging Face – Optimum Quantization Concept Guide

Medium (I. Sanghao) – What is Quantization and Why It Matters for Inference

IBM – Quantization

GoPenAI – Optimizing AI Models with Quantization

Case Study: Real-Time Drone Detection on the AMB82-Mini

Kaggle – YOLO Drone Detection Dataset

WongKinYiu/yolov7 (GitHub)

Ameba Arduino SDK Docs – Object Detection (ObjectDetectionLoop)

Ameba Arduino SDK Docs – Customized AI Model Installation Guide

amebaiot.com – AmebaPro2 AI Convert Model

amebaiot.com – AmebaPro2 Apply AI Model Docs

amebaiot.com – Offline AI Model Conversion

How2Electronics – Object Detection & Identification with AMB82-Mini

Ameba Forum – First Upload / AI Conversion

Ameba Forum – AMB82-Mini Customized AI Model (missing model file)

Case Study: Real-Time Drone Detection on the Raspberry Pi Zero 2 W

Raspberry Pi setup guide (MLSysBook)

Raspberry Pi Imager / getting started

YOLOv7 (export tooling)

NCNN — using NCNN with PyTorch/ONNX

NCNN — build instructions

Raspberry Pi forums — camera/libcamera discussion

Raspberry Pi camera software documentation

Benchmarking and Performance Comparison

Ameba Arduino AIoT Documentation — disabling NN runtime debug logs (tick/FPS messages)
Ameba IoT Forum — "AMB82-Mini Arduino monitor filled with unwanted debug messages 'YOLOv7t_SD tick'" (confirms the same tick[N]/FPS log pattern from the AMB82-Mini's YOLOv7-tiny example)
Ultralytics YOLO11 documentation
Ultralytics NCNN export documentation

Drone Fundamentals and Lab Visit

UAVMODEL — The Ultimate Guide to UAV Drones: Technology, Components, and Industrial Applications
ZenaTech — Drone Technology Basics: A Beginner's Guide
ZenaTech — How Drone Controllers Work: A Beginner's Guide
Netline — How Drones Are Being Controlled and the Way to Neutralise Them
Instructables — How to Build the Fastest Quadcopter in 3 Hours
ArduPilot — Mission Planner overview
OpenHD FPV — Mission Planner ground station software
Pilot Institute — Drone IMU Calibration Explained
DragonPlate — What Is Carbon Fiber?
Prototek — Drone Manufacturing: A Guide from Components to Production